

**A MAJOR PROJECT REPORT ON**  
**REAL-TIME PEDESTRIAN DETECTION USING**  
**YOLOV8 WITH OPENVINO OPTIMIZATION**

Submitted in partial fulfilment of the requirement for the award of degree

**BACHELOR OF TECHNOLOGY**  
**IN**  
**COMPUTER SCIENCE AND ENGINEERING**  
**(ARTIFICIAL INTELLIGENCE AND MACHINE**  
**LEARNING)**

**by**

**P. SAI SRIKAR – 22671A6631**

**P. SAIDEEP – 22671A6651**

**B. SHRUTI – 22671A6653**

**Under the esteemed guidance of**

**MR. MD. MAHEBUB ALI**  
**ASSISTANT PROFESSOR**



**DEPARTMENT OF COMPUTER SCIENCE AND**  
**ENGINEERING (ARTIFICIAL INTELLIGENCE AND**  
**MACHINE LEARNING)**

**J.B INSTITUTE OF ENGINEERING & TECHNOLOGY**  
**(UGC AUTONOMOUS)**

**(Accredited by NAAC & NBA, Approved by AICTE & Permanently affiliated by**  
**JNTUH) Yenkapally, Moinabad mandal, R.R. Dist-75(TG)**

**(2022-2026)**

**J.B INSTITUTE OF ENGINEERING & TECHNOLOGY  
(UGC AUTONOMOUS)**

**(Accredited by NAAC & NBA, Approved by AICTE & Permanently affiliated by  
JNTUH) Yenkapally, Moinabad mandal, R.R. Dist-75(TG)**

**DEPARTMENT OF COMPUTER SCIENCE AND  
ENGINEERING (ARTIFICIAL INTELLIGENCE AND  
MACHINE LEARNING)**



**CERTIFICATE**

This is to certify that this project report entitled “**REAL-TIME PEDESTRIAN DETECTION USING YOLOV8 WITH OPENVINO OPTIMIZATION**”, by **P. SAI SRIKAR (22671A6631), P. SAIDEEP (22671A6651) & B.SHRUTI (22671A6653)** submitted in partial fulfilment of the requirements for the degree of **Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)** of the J.B Institution of Engineering and Technology Hyderabad, during the academic year 2025-2026, is a Bonafide record of work carried out under my guidance and supervision.

**INTERNAL GUIDE**  
**MR. MD MAHEBUB ALI**  
**ASSISTANT PROFESSOR**

**HEAD OF THE DEPARTMENT**  
**DR. GURRAMPALLY KUMAR**  
**ASSOCIATE PROFESSOR**

**EXTERNAL EXAMINER**

**J.B INSTITUTE OF ENGINEERING & TECHNOLOGY**  
**(UGC AUTONOMOUS)**

(Accredited by NAAC & NBA, Approved by AICTE & Permanently affiliated by  
JNTUH) Yenkapally, Moinabad mandal, R.R. Dist-75(TG)

**DEPARTMENT OF COMPUTER SCIENCE AND  
ENGINEERING (ARTIFICIAL INTELLIGENCE AND  
MACHINE LEARNING)**



**DECLARATION**

We hereby declare that this project work entitled “**REAL-TIME PEDESTRIAN DETECTION USING YOLOV8 WITH OPENVINO OPTIMIZATION**” carried out under the guidance of, **MR. MD MAHEBUB ALI, Assistant Professor** is submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)**. This is a record of Bonafide work carried out by us and the results embodied in this Major Project report have not been reproduced or copied from any source. The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree.

**P. SAI SRIKAR – 22671A6631**

**P. SAIDEEP – 22671A6651**

**B. SHRUTI - 22671A6653**

## ACKNOWLEDGEMENT

At outset I express my gratitude to Almighty lord for showering his grace and blessings upon me to complete this Major Project. Although my name on the cover of this book, many people have contributed in some form or the other to this project development. I could not have done this Project without the assistance or support of each of the following.

First of all, I am highly indebted to **Dr. P.C. KRISHNAMACHARYA, Principal** for giving us the permission to carry out this Major Project.

I would like to thank **Dr. G. KUMAR, Associate Professor & Head of the Department** of Computer Science and Engineering (Artificial Intelligence and Machine Learning), for being moral support throughout the period of the study in the Department.

I am grateful to **Dr. AKHEEL MOHAMMED, Associate Professor & Project Coordinator**, Computer Science and Engineering (Artificial Intelligence and Machine Learning), for his valuable suggestions and guidance given by him during the execution of this Project work.

I am grateful to **MR. MD MAHEBUB ALI, Assistant Professor**, Computer Science and Engineering (Artificial Intelligence and Machine Learning), for his valuable suggestions and guidance given by him during the execution of this Project work.

I would like to thank the Teaching and non-Teaching staff of the Department of Artificial Intelligence and Machine Learning for sharing their knowledge with us.

**P. SAI SRIKAR – 22671A6631**

**P. SAIDEEP – 22671A6651**

**B. SHRUTI - 22671A6653**

# ABSTRACT

This project presents the design and implementation of a real-time pedestrian detection system using YOLOv8 with OpenVINO optimization, focusing on achieving high accuracy and low latency in dynamic environments. Pedestrian detection plays a crucial role in applications such as intelligent surveillance, autonomous driving, and smart traffic management, where accurate and timely identification of humans is essential. Traditional computer vision approaches often face challenges such as varying lighting conditions, occlusions, and complex backgrounds, limiting their effectiveness in real time scenarios. To overcome these limitations, this project utilizes YOLOv8, a state-of-the-art deep learning-based object detection model known for its speed and precision. The model is trained on datasets containing diverse pedestrian scenarios, enabling it to perform robust detection across different environments. The system accepts input in the form of images and video streams and identifies pedestrians by generating bounding boxes along with confidence scores. To further enhance performance, OpenVINO optimization is applied, significantly improving inference speed, especially on CPU based systems, thereby making the solution more efficient and cost-effective for practical deployment. Experimental results demonstrate that the proposed system achieves high detection accuracy while maintaining real-time performance, even in challenging conditions involving multiple pedestrians and partial occlusions. The incorporation of OpenVINO reduces computational overhead and increases frame processing speed without compromising accuracy. In conclusion, this project demonstrates the effectiveness of combining deep learning techniques with optimization frameworks to develop a reliable and efficient pedestrian detection system, contributing to advancements in computer vision and real-time intelligent applications.

**Index Terms** - Pedestrian Detection, YOLOv8, OpenVINO, KITTI Dataset, Object Detection, Deep Learning, Convolutional Neural Networks, Real-Time Inference, ONNX, Model Optimization, Edge AI, CPU-Based Deployment, Computer Vision, mAP, FPS.

# CONTENTS

| DESCRIPTION                          | PAGE NO      |
|--------------------------------------|--------------|
| Abstract.....                        | i            |
| Table of Contents.....               | ii           |
| List Of Figures.....                 | iv           |
| List Of Tables.....                  | v            |
| <b>1. Introduction.....</b>          | <b>1-4</b>   |
| 1.1 Introduction.....                | 1            |
| 1.2 Background and Motivation.....   | 1            |
| 1.3 Problem definition.....          | 2            |
| 1.4 Objectives of The Study.....     | 2            |
| 1.5 Scope of the Project.....        | 3            |
| 1.6 Significance of the Study.....   | 4            |
| <b>2. Literature Survey.....</b>     | <b>5 -7</b>  |
| <b>3. System Analysis.....</b>       | <b>8-20</b>  |
| 3.1 Existing Systems.....            | 8            |
| 3.2 Proposed System.....             | 11           |
| 3.3 System Requirements.....         | 14           |
| 3.4 System Modules.....              | 18           |
| <b>4. System Design.....</b>         | <b>21-29</b> |
| 4.1 Architecture.....                | 21           |
| 4.2 Detailed Components.....         | 21           |
| 4.3 Workflow.....                    | 22           |
| 4.4 UML Diagrams.....                | 24           |
| <b>5. System Implementation.....</b> | <b>30-75</b> |
| 5.1 Technologies.....                | 30           |
| 5.2 Source Code.....                 | 34           |
| 5.3 Output Screenshots.....          | 71           |

|                                                   |              |
|---------------------------------------------------|--------------|
| <b>6. System Testing.....</b>                     | <b>76-82</b> |
| 6.1 Testing Overview.....                         | 76           |
| 6.2 Unit Testing.....                             | 77           |
| 6.3 Integration Testing.....                      | 78           |
| 6.4 White Box Testing.....                        | 79           |
| 6.5 Black Box Testing.....                        | 80           |
| 6.6 Performance Testing.....                      | 81           |
| <b>7. Conclusion And Future Enhancements.....</b> | <b>83-84</b> |
| 7.1 Conclusion.....                               | 83           |
| 7.2 Future Enhancements.....                      | 84           |
| <b>Bibliography.....</b>                          | <b>85-86</b> |

## LIST OF FIGURES

| <b>Figure No.</b> | <b>Title of the Figure</b>                           | <b>Page No.</b> |
|-------------------|------------------------------------------------------|-----------------|
| Figure 1.1        | Bounding boxes around pedestrians.....               | 1               |
| Figure 1.2        | Object Detection Using YOLO.....                     | 4               |
| Figure 3.1        | YOLO Version-6 Architecture.....                     | 10              |
| Figure 3.2        | YOLO Version-8 Architecture.....                     | 14              |
| Figure 4.1        | Model architecture of YOLO version-8.....            | 21              |
| Figure 4.3        | Data workflow diagram.....                           | 23              |
| Figure 4.4.1      | Use Case Diagram.....                                | 25              |
| Figure 4.4.2      | Class Diagram.....                                   | 26              |
| Figure 4.4.3      | Activity Diagram.....                                | 27              |
| Figure 4.4.3      | Sequence Diagram.....                                | 28              |
| Figure 4.4.4      | Component Diagram.....                               | 29              |
| Figure 5.3.1      | Image Detection Interface.....                       | 72              |
| Figure 5.3.2      | Video Detection Interface.....                       | 72              |
| Figure 5.3.3      | Dataset Browser Displaying KITTI Images.....         | 73              |
| Figure 5.3.4      | Pedestrian Detection Output with Bounding Boxes..... | 73              |
| Figure 5.3.5      | Dataset Browser Detection Result.....                | 74              |
| Figure 5.3.6      | Training and Validation Performance Metrics.....     | 74              |
| Figure 5.3.7      | Precision–Recall Curve of YOLOv8 Model.....          | 75              |
| Figure 5.3.8      | Performance Comparison of YOLOv8.....                | 75              |



## LIST OF TABLES

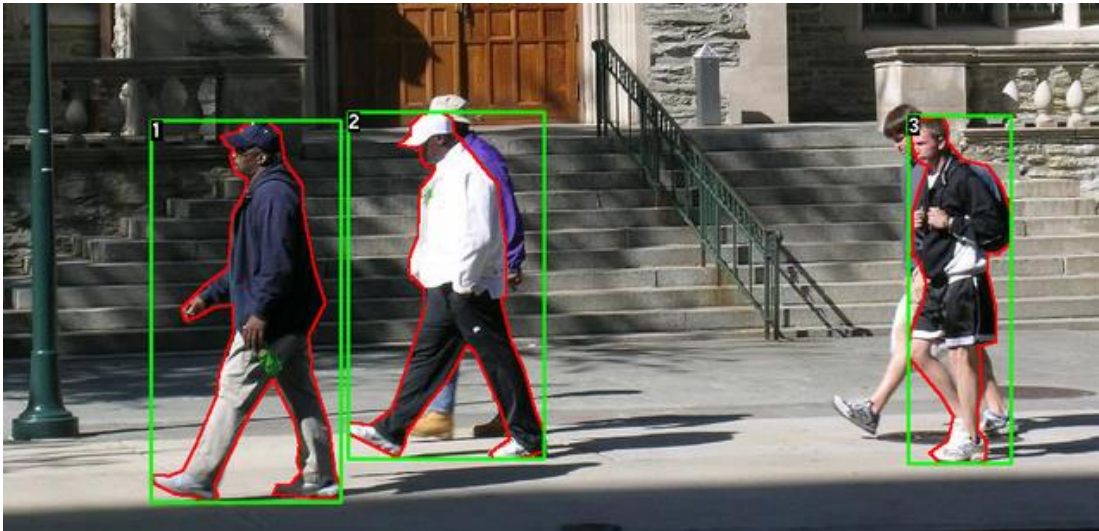
| <b>Table No.</b> | <b>Title of the Table</b> | <b>Page No.</b> |
|------------------|---------------------------|-----------------|
| Table 5.3        | Input Types.....          | 71              |
| Table 6.1        | Unit Testing.....         | 78              |
| Table 6.2        | Integration Testing.....  | 79              |
| Table 6.3        | White Box Testing.....    | 80              |
| Table 6.4        | Black Box Testing.....    | 81              |
| Table 6.5        | Performance Testing.....  | 82              |

# CHAPTER - 1

## INTRODUCTION

### 1.1 INTRODUCTION

Pedestrian detection is essential in computer vision applications such as autonomous driving, surveillance, and smart city systems, where accurate real-time identification is crucial for safety and decision-making. Traditional methods struggle with challenges like occlusion, lighting variations, and crowded scenes, limiting their reliability. With advancements in deep learning, YOLOv8 emerges as a powerful solution, offering high accuracy, speed, and robust feature extraction. This project leverages YOLOv8 to build an efficient pedestrian detection system capable of handling diverse real-world scenarios and improving overall detection performance.



**Figure 1.1 Bounding boxes around pedestrians**

### 1.2 BACKGROUND AND MOTIVATION

Pedestrian detection is a critical component of intelligent transportation systems, advanced driver assistance systems (ADAS), and modern surveillance solutions. With increasing urbanization and vehicle density, pedestrian safety has become a major concern worldwide. Traditional computer vision techniques such as Haar Cascades and Histogram of Oriented Gradients (HOG) combined with Support Vector Machines (SVM) provided early solutions but struggled with complex backgrounds, occlusion, and varying lighting conditions. The emergence of deep learning, particularly Convolutional Neural Networks (CNNs), has significantly

improved object detection accuracy and real-time performance. YOLOv8, a state-of-the-art single-stage object detection model, offers high detection speed and improved precision, making it suitable for real-time pedestrian detection tasks. Furthermore, efficient deployment on CPU based systems remains a challenge, which motivates the integration of OpenVINO for optimized edge inference.

### **1.3 PROBLEM DEFINITION**

While pedestrian detection systems are widely researched and implemented, many existing solutions primarily focus on achieving high accuracy without considering deployment efficiency. Most high-performance deep learning models require GPU based hardware to achieve real-time performance, which increases cost and power consumption. In many real-world applications, especially edge devices and embedded systems, GPU availability may be limited or absent. Therefore, there exists a need for a system that balances detection accuracy with optimized CPU-based inference performance. The problem addressed in this project is to develop a pedestrian detection framework that not only achieves reliable detection using YOLOv8 but also ensures efficient real-time performance through OpenVINO optimization, making it suitable for deployment in resource-constrained environments.

### **1.4 OBJECTIVES OF THE STUDY**

The primary objective of this project is to design and develop a real-time pedestrian detection system using the YOLOv8 deep learning model and optimize it for CPU based deployment using OpenVINO.

The specific objectives include:

1. To collect and pre-process the KITTI dataset for pedestrian detection.
2. To convert dataset annotations into YOLO format for model training.
3. To fine-tune a pre-trained YOLOv8 model for single-class pedestrian detection.
4. To evaluate the trained model using performance metrics such as Precision, Recall, mean Average Precision (mAP), and Frames Per Second (FPS).
5. To export the trained model into ONNX format for interoperability. To convert and optimize the model using OpenVINO's Intermediate Representation (IR) format.
6. To deploy the optimized model for real-time CPU-based inference.

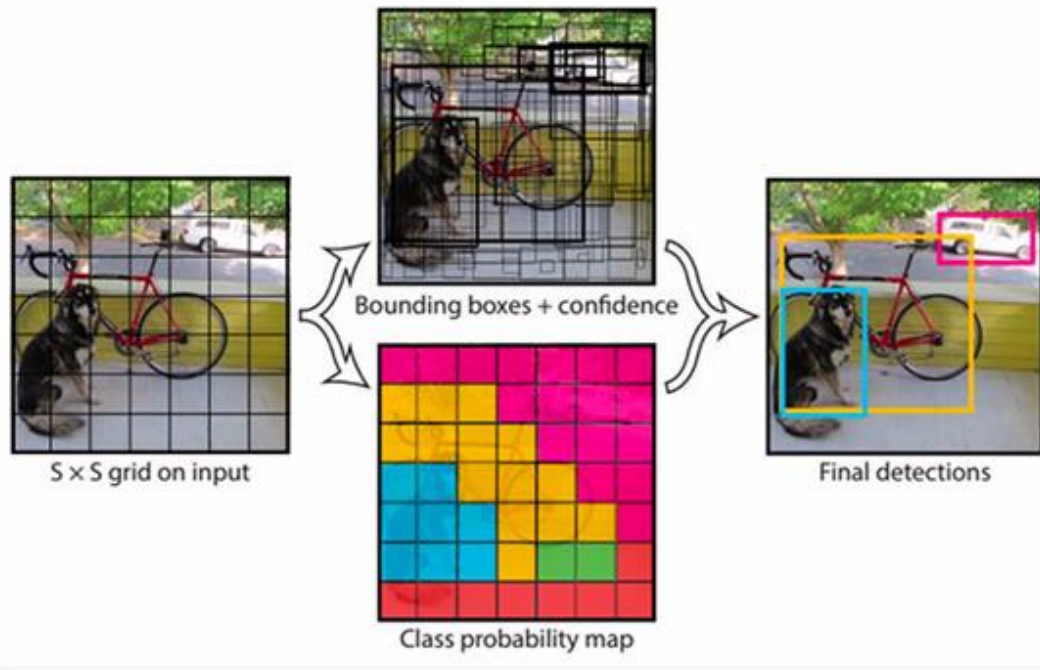
These objectives aim to ensure both detection accuracy and efficient deployment.

## **1.5 SCOPE OF THE PROJECT**

The scope of this project includes dataset pre-processing, training and evaluation of the YOLOv8 model for pedestrian detection, and optimization using OpenVINO for CPU based inference. The project focuses specifically on single-class pedestrian detection using the KITTI dataset. It involves model export to ONNX format and conversion to OpenVINO's IR format to enhance inference speed. The system is designed to operate offline and does not depend on cloud-based processing. However, the project does not include full autonomous vehicle integration, multi-sensor fusion (such as LiDAR or radar), hardware-level embedded implementation, or multi-class object detection. Future work may extend the system to embedded platforms, multi-class detection, or integration with intelligent traffic control systems.

## **1.6 SIGNIFICANCE OF THE STUDY**

This study demonstrates the practical integration of deep learning-based object detection with hardware-level optimization techniques for real-world deployment. By combining YOLOv8 with OpenVINO optimization, the system achieves improved inference speed on CPU devices while maintaining competitive detection accuracy. The significance of this project lies in bridging the gap between research-oriented model development and deployment-ready solutions. It highlights the importance of performance evaluation, model conversion, and optimization in building scalable and cost-effective AI systems. The proposed framework can contribute to intelligent transportation systems, surveillance applications, and edge AI deployments, emphasizing the growing relevance of optimized real-time computer vision systems in modern technological environments. Furthermore, the approach ensures efficient utilization of computational resources, making it suitable for deployment in resource-constrained environments. It also provides a flexible foundation for integrating future advancements in deep learning and real-time analytics. In addition, the system promotes faster decision-making in safety-critical applications by delivering timely detection results. This adaptability makes it a promising solution for continuous improvements and large-scale real-world implementation



**Figure 1.2 Object Detection Using YOLO**

## **CHAPTER – 2**

### **LITERATURE REVIEW**

#### **2.1 YOU ONLY LOOK ONCE: UNIFIED, REAL-TIME OBJECT DETECTION**

**AUTHORS:** J. Redmon, S. Divvala, R. Girshick, and A. Farhadi

Object detection is a fundamental problem in computer vision, and traditional approaches often rely on complex pipelines involving region proposals and classification. In this paper, the authors propose YOLO (You Only Look Once), a unified model that performs object detection in a single forward pass of a neural network. This approach significantly improves detection speed while maintaining competitive accuracy. YOLO divides the image into grids and predicts bounding boxes along with class probabilities directly. The model is capable of real-time detection and performs well in general object detection tasks. This work laid the foundation for modern real-time object detection systems and influenced the development of advanced models like YOLOv8 used in this project.

#### **2.2 FASTER R-CNN: TOWARDS REAL-TIME OBJECT DETECTION WITH REGION PROPOSAL NETWORKS**

**AUTHORS:** S. Ren, K. He, R. Girshick, and J. Sun

This paper introduces Faster R-CNN, an advanced region-based object detection framework that integrates region proposal generation with deep learning. The model uses a Region Proposal Network (RPN) to generate candidate object regions, which are then classified using convolutional neural networks. Faster R-CNN significantly improves detection accuracy compared to earlier methods such as R-CNN and Fast R-CNN. However, the model is computationally intensive and requires high processing power, making it less suitable for real-time applications. Despite its limitations, this approach plays a crucial role in the evolution of object detection techniques.

#### **2.3 YOLOV4: OPTIMAL SPEED AND ACCURACY OF OBJECT DETECTION**

**AUTHORS:** A. Bochkovskiy, C. Y. Wang, and H. Y. M. Liao

The YOLOv4 model improves upon previous YOLO versions by introducing enhancements in feature extraction, training strategies, and network architecture. It

incorporates techniques such as CSPDarknet, spatial pyramid pooling, and data augmentation to improve detection accuracy while maintaining real-time speed. The model demonstrates high performance in detecting objects under complex conditions, including varying lighting and backgrounds. YOLOv4 serves as an important step toward achieving a balance between speed and accuracy in object detection tasks.

## **2.4 YOLOV8: ADVANCED REAL-TIME OBJECT DETECTION MODEL**

**AUTHORS:** Ultralytics

YOLOv8 represents the latest advancement in the YOLO series, offering improved performance in terms of speed, accuracy, and flexibility. The model uses an anchor-free detection approach and enhanced feature extraction techniques to improve detection efficiency. It is capable of handling multiple objects, varying scales, and complex environments effectively. YOLOv8 is highly optimized for real-time applications and supports easy integration with deployment frameworks. Due to its superior performance, it is widely used in applications such as pedestrian detection, surveillance, and autonomous driving systems.

## **2.5 KITTI VISION BENCHMARK SUITE**

**AUTHORS:** A. Geiger, P. Lenz, and R. Urtasun

The KITTI dataset is a widely used benchmark for computer vision tasks related to autonomous driving. It contains real-world images captured from vehicle-mounted cameras in urban environments. The dataset includes annotations for various object classes such as pedestrians, vehicles, and cyclists. Its diversity in terms of lighting, occlusion, and background complexity makes it highly suitable for training and evaluating object detection models. In this project, the KITTI dataset is used to train the pedestrian detection model, ensuring robustness and real-world applicability.

## **2.6 OPENVINO TOOLKIT FOR DEEP LEARNING OPTIMIZATION**

**AUTHORS:** Intel Corporation

OpenVINO (Open Visual Inference and Neural Network Optimization) is a toolkit designed to optimize deep learning models for efficient inference on Intel hardware. It converts trained models into an intermediate representation and applies hardware-specific optimizations to improve speed and reduce latency. This enables real-time performance on CPU-based systems without requiring high-end GPUs.

OpenVINO plays a critical role in this project by enhancing inference speed and making the system suitable for deployment in resource-constrained environments.

## **2.7 YOLOv8 FOR OBJECT DETECTION**

**AUTHORS:** Ultralytics

YOLOv8 (You Only Look Once version 8) is a state-of-the-art real-time object detection algorithm known for its speed and accuracy. It follows a single-stage detection approach, where object localization and classification are performed simultaneously in one pass through the network. This makes it highly efficient for real-time applications such as surveillance and autonomous driving. YOLOv8 introduces improvements in model architecture, including better feature extraction and anchor-free detection mechanisms. It also supports tasks like object detection, segmentation, and classification within a unified framework. The model is optimized for both high accuracy and low computational cost. In this project, YOLOv8 is used to detect pedestrians in images and video streams, providing fast and reliable results. Its ability to perform well under challenging conditions such as occlusion and varying lighting makes it suitable for real-world deployment.

## **2.8 DEEP LEARNING IN COMPUTER VISION**

**AUTHORS:** Yann LeCun, Geoffrey Hinton, and Yoshua Bengio

Deep learning has revolutionized computer vision by enabling machines to automatically learn features from data without manual feature engineering. Convolutional Neural Networks (CNNs) are the backbone of most vision systems, capable of extracting hierarchical features from images. These models have significantly improved the performance of tasks such as image classification, object detection, and segmentation. The use of large datasets and powerful GPUs has further accelerated advancements in this field. Deep learning models can generalize well across different environments when trained with diverse data. In this project, deep learning techniques are used to build an efficient pedestrian detection system. The combination of CNN-based models and optimization frameworks ensures high accuracy and real-time performance. This makes deep learning a key technology for modern intelligent systems.



## **CHAPTER – 3**

### **SYSTEM ANALYSIS**

#### **3.1 EXISTING SYSTEMS**

##### **3.1.1 Traditional Handcrafted Feature-Based Systems**

Before the advent of deep learning, pedestrian detection heavily relied on handcrafted features engineered to capture edge patterns, textures, and shapes that differentiate humans from the background.

##### **a) Histogram of Oriented Gradients (HOG) + SVM**

The HOG descriptor extracts directional gradients from an image, capturing the contour and silhouette of pedestrians. These features are then passed to a Support Vector Machine (SVM) classifier to determine the presence of a pedestrian.

##### **Advantages:**

1. Relatively simple and computationally inexpensive.
2. Works well for upright, full-body pedestrian images.

##### **Limitations:**

1. Poor performance under partial occlusion or varying poses. Sensitive to illumination changes.
2. Cannot operate reliably in real-time for high-resolution video streams.
3. Very low accuracy compared to modern CNN-based methods.

This approach marked the beginning of automated pedestrian detection but was soon replaced by more powerful feature extractors.

##### **3.1.2 Classical Machine Learning Detectors**

Handcrafted features evolved further with machine learning classifiers to improve robustness.

##### **a) Haar Cascades**

Haar-like features combined with AdaBoost classifiers were used extensively in earlier computer vision applications. Though fast, Haar cascades are highly sensitive to variations in appearance and struggle with complex backgrounds.

##### **b) Integral Channel Features (ICF)**

ICF and aggregated channel features (ACF) introduced multiple image channels (color, gradients, etc.) to strengthen detection. Although more efficient than HOG, they still fail to generalize well in diverse environments.

**Limitations of classical ML systems:**

1. Limited ability to learn high-level semantic features.
2. Require careful tuning and feature engineering.
3. Prone to high false-positive rates.
4. Weak performance in crowded scenes and occlusions.

**3.1.3 Deep Learning-Based Pedestrian Detection**

Deep learning revolutionized pedestrian detection through automatic feature learning.

**a) Region-Based Convolutional Neural Networks (R-CNN, Fast R-CNN, Faster RCNN)**

These models generate region proposals and classify each region using CNNs.

**Advantages:**

1. Higher accuracy than handcrafted models.
2. Better understanding of complex human shapes.

**Limitations:**

1. Slow inference time (not suitable for real-time applications).
2. Heavy computational resources required.
3. Region proposal step introduces latency.

**b) Single Shot Detectors (SSD)**

SSD introduced real-time detection capability by eliminating the region proposal step. It directly predicts bounding boxes and class probabilities.

**Strengths:**

1. Faster than R-CNN variants.
2. Decent accuracy on large objects.

**Weaknesses:**

1. Lower accuracy for small pedestrians.
2. Struggles in low-contrast environments.

### c) YOLO (You Only Look Once) Family

YOLO models unified detection into a single forward pass, enabling real-time performance. Versions from YOLOv1 to YOLOv7 significantly improved accuracy, speed, and detection performance.

#### Advantages:

1. High FPS (frames per second).
2. Great for real-time systems such as autonomous vehicles.

#### Limitations in older YOLO versions:

1. Struggled with small object detection.
2. Limited feature learning compared to transformer-based models.
3. Higher false negatives in crowded scenes

#### 3.1.4 Limitations in Existing Systems

Despite vast improvements, existing systems still face multiple issues:

1. Lower accuracy for small or partially occluded pedestrians.
2. Poor generalization under varying lighting and weather conditions.
3. Computational inefficiency for high-resolution real-time video.

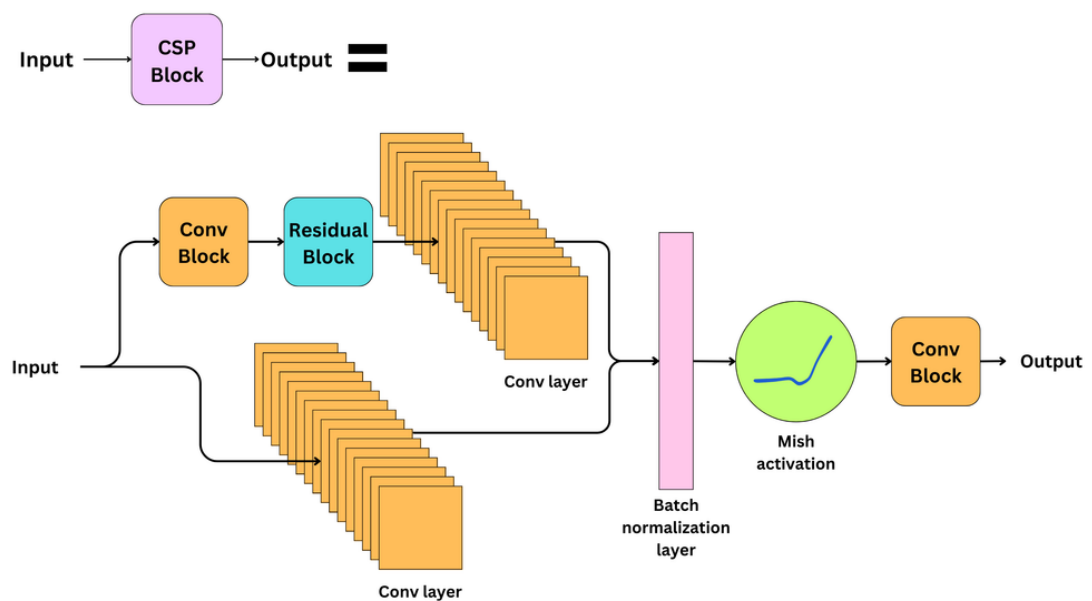


Figure 3.1 YOLO Version-6 Architecture

## 3.2 PROPOSED SYSTEM

The proposed system introduces a modern, robust, and deployment-oriented pedestrian detection framework using the YOLOv8 deep learning model integrated with Intel’s OpenVINO optimization toolkit. YOLOv8, developed by Ultralytics, represents a significant advancement over previous YOLO versions in terms of backbone architecture, detection head design, and training efficiency. However, unlike conventional YOLO implementations that focus solely on detection accuracy, the proposed system enhances the pipeline by incorporating model conversion and CPU based optimization for real-time edge deployment.

The system follows a structured workflow that includes dataset preprocessing, annotation handling, model training, validation, evaluation, ONNX export, OpenVINO conversion, and optimized inference. The primary objective of the proposed system is to achieve high detection accuracy, low latency, improved CPU efficiency, and strong generalization across diverse pedestrian scenarios without relying on GPU hardware.

### 3.2.1 Modern Deep Learning-Based Architecture

The proposed system leverages YOLOv8’s advanced neural architecture, which combines a CSP-inspired backbone with an optimized Path Aggregation Network (PAN) for multi-scale feature extraction. YOLOv8 introduces a decoupled detection head where classification and bounding box regression are processed separately, improving detection stability and accuracy.

#### Key Architectural Strengths:

1. Lightweight and scalable variants (YOLOv8n, s, m, l, x).
2. Improved detection of small and partially occluded pedestrians.
3. Enhanced objectness learning for better precision and recall.
4. Efficient single-stage detection for real-time processing.
5. Built-in Non-Maximum Suppression (NMS) improvements to reduce redundant detections.

Unlike earlier systems, this architecture serves as the foundation before optimization using OpenVINO for improved CPU inference.

### 3.2.2 Dataset Preprocessing and Augmentation

The system utilizes the KITTI dataset for pedestrian detection. Several preprocessing steps are applied to ensure compatibility and performance:

1. Normalization of pixel values for consistent model input.
2. Image resizing to YOLOv8's required input dimensions (e.g., 640×640).
3. Conversion of labels into YOLO annotation format.
4. Train-validation split to ensure model generalization.
5. Optional data augmentation (scaling, flipping, brightness adjustment) to improve robustness.

These preprocessing techniques enhance the model's ability to generalize across diverse real-world road scenarios.

### 3.2.3 Training and Evaluation Pipeline

The training pipeline includes:

1. Selection of an appropriate YOLOv8 model variant (e.g., YOLOv8n for lightweight deployment).
2. Configuration of hyperparameters such as batch size, epochs, and learning rate.
3. Use of early stopping and learning rate scheduling.
4. Performance evaluation using:
  - Precision
  - Recall
  - mAP@50
  - mAP@50-90
  - Frames Per Second (FPS)

The best-performing model weights are saved for further optimization. This structured training methodology ensures strong detection performance while preventing overfitting.

### 3.2.4 Model Conversion and OpenVINO Optimization

To enhance deployment efficiency, the trained YOLOv8 model is exported to ONNX format. The ONNX model is then converted into OpenVINO's Intermediate Representation (IR) format (.xml and .bin files). OpenVINO optimizes the model for Intel CPU architectures by:

1. Layer fusion and graph optimization.
2. Memory usage reduction. Efficient scheduling of inference operations.
3. Hardware-specific acceleration.

This optimization significantly improves CPU-based inference speed and reduces latency while maintaining detection accuracy. A performance comparison is conducted to measure FPS before and after OpenVINO optimization.

### **3.2.5 Real-Time Detection Integration**

The optimized model is deployed for real-time pedestrian detection using:

1. Live camera feed processing.
2. Video stream analysis.
3. Real-time bounding box visualization with confidence scores.
4. Display of FPS for performance monitoring.

The system operates offline and does not require cloud connectivity, making it suitable for edge AI deployment in resource-constrained environments.

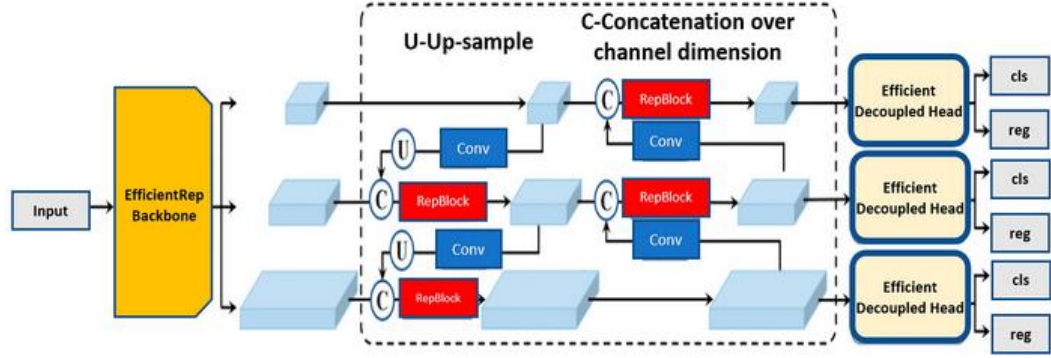
Potential Real-Time Use Cases:

1. Intelligent transportation systems.
2. Pedestrian safety monitoring.
3. Traffic surveillance systems.
4. Edge-based smart camera solutions.

### **3.2.6 Advantages of the Proposed YOLOv8 + OpenVINO System**

1. High Accuracy: Improved detection performance through YOLOv8 architecture.
2. CPU-Based Real-Time Performance: OpenVINO optimization increases FPS without GPU dependency.
3. Reduced Hardware Cost: Eliminates need for expensive GPU hardware.
4. Edge AI Compatibility: Suitable for deployment on local systems.
5. Improved Efficiency: Balanced trade-off between accuracy and computational speed.
6. Deployment-Oriented Architecture: Includes full pipeline from training to optimized inference.

By integrating advanced deep learning detection with hardware-level CPU optimization, the proposed system provides a practical, scalable, and industry-relevant solution that outperforms traditional and non-optimized deep learning approaches in real-world deployment scenarios.



**Figure 3.2 YOLO Version-8 Architecture**

### 3.3 SYSTEM REQUIREMENTS

System requirements define the essential conditions, capabilities, and constraints that the pedestrian detection system must satisfy to function efficiently. These requirements ensure that the system operates reliably in real-time environments, delivers accurate detection results, and integrates seamlessly with existing hardware and software infrastructures. This section outlines the functional requirements, non-functional requirements, as well as the hardware and software requirements necessary for implementing the proposed YOLOv8-based pedestrian detection system.

#### 3.3.1 Functional Requirements

##### 1. Data Input and Preprocessing

- a) The system shall accept raw input data in the form of images, offline video files, or real-time video streams from a connected camera.
- b) The system shall perform preprocessing operations such as resizing, normalization, and data augmentation. These processes help improve the robustness of the model.
- c) The system shall support input datasets annotated in YOLO-compatible formats. If annotation formats differ, the system shall convert them accordingly.

## 2. Pedestrian Detection and Classification

- a) The system shall detect pedestrians within an image or video stream using the trained YOLOv8 model.
- b) The system shall generate bounding boxes around detected pedestrians and assign confidence scores to each detection.
- c) The system shall process video or camera input in real time with minimal latency, enabling timely detection and response.
- d) The system shall allow the user to configure detection sensitivity through confidence and IoU thresholds.

## 3. Training and Model Optimization

- a) The system shall support training on custom datasets or fine-tuning pre-trained YOLOv8 models.
- b) The system shall provide the ability to set training parameters such as learning rate, batch size, epochs, and augmentation policy.
- c) The system shall generate metrics such as mAP (Mean Average Precision), precision, recall, and loss curves during training and validation.
- d) The system shall save the best-performing model weights automatically.

## 4. Output Visualization and Reporting

- a) The system shall display bounding boxes, labels, and confidence levels directly on the output image or video.
- b) The system shall store or present detection results frame-by-frame for evaluation and analysis.
- c) The system shall maintain logs of detection events, including time, bounding box coordinates, and confidence scores.
- d) The system may provide a minimal interface for loading videos, adjusting parameters, and viewing results.

## 5. Storage and Exporting

- a) The system shall support exporting trained models in various formats such as ONNX, TorchScript, or TensorRT.
- b) The system shall allow saving processed video frames or annotated images.
- c) The system shall store datasets and annotations in a structured directory required by YOLOv8



### 3.3.2 Non-Functional Requirements

#### 1. Performance Requirements

- a) The system shall achieve a minimum of 20–30 FPS for real-time pedestrian detection depending on the GPU or CPU available.
- b) The system shall achieve a high mAP score for pedestrian class detection. This ensures reliable performance under diverse scenarios.
- c) Detection and response time shall be minimized to prevent delays in critical use cases like autonomous driving or surveillance.

#### 2. Reliability and Robustness

- a) The system shall consistently detect pedestrians even under varying lighting, occlusion, and movement.
- b) The system shall handle errors (e.g., missing frames, corrupted data) without crashing.
- c) When encountering invalid input formats, the system shall display appropriate warnings rather than terminating unexpectedly.

#### 3. Usability Requirements

- a) If a UI is provided, it shall be simple and intuitive for users without technical expertise.
- b) Detected bounding boxes shall be clearly visible, with readable labels and confidence values.
- c) The system shall include documentation on installation, dataset preparation, training procedures, and deployment.

#### 4. Scalability

- a) The system shall support multiple versions of YOLOv8 (nano, small, medium, large, xlarge) based on hardware capability.
- b) The system shall handle large datasets without performance degradation.
- c) The system shall allow deployment on edge devices (NVIDIA Jetson, Raspberry Pi with accelerators) and cloud servers.
- d) The system shall provide real-time inference with optimized latency and high FPS to ensure efficient pedestrian detection in live environments.

- e) The system shall support model optimization techniques such as quantization (e.g., INT8) and hardware-specific acceleration (e.g., OpenVINO, TensorRT) to improve performance on CPU and edge devices.

### **3.3.3 Hardware Requirements**

#### **1. Minimum Hardware Requirements**

- a) Processor: Intel Core i5 (8th Generation or higher) / AMD Ryzen 5
- b) RAM: 8 GB Storage: 25 GB free space (for dataset, model weights, and OpenVINO files)
- c) GPU: Optional (Not mandatory; CPU-based inference supported using OpenVINO)

These specifications are sufficient for lightweight YOLOv8 training (e.g., YOLOv8n) and real-time CPU-based inference after OpenVINO optimization.

#### **2. Recommended Hardware Requirements**

- a) Processor: Intel Core i7 / i9 (recommended for better OpenVINO acceleration)
- b) RAM: 16 GB Storage: SSD with at least 100 GB free space
- c) GPU: NVIDIA RTX series (optional, recommended only for faster training)
- d) High-Resolution Camera: For accurate real-time pedestrian detection

For larger datasets or faster experimentation, higher RAM and GPU support significantly reduce training time, although GPU is not required for final deployment.

### **3.3.4 Software Requirements**

#### **1. Operating System**

- a) Windows 10 / Windows 11
- b) Ubuntu 20.04 or later (recommended for deep learning and OpenVINO compatibility)
- c) macOS (supported for inference, limited training support)

#### **2. Programming Languages and Frameworks**

- a) Python 3.8+ – Primary programming language for model development and deployment.
- b) PyTorch – Backend framework used by YOLOv8 for training and inference.

- c) Ultralytics YOLOv8 Library – Core framework for training, validation, and detection.
- d) OpenVINO Toolkit – Used for model optimization and CPU-based inference acceleration.
- e) ONNX (Open Neural Network Exchange) – Intermediate format for model export and interoperability.

### 3. Supporting Libraries

- a) NumPy – Numerical operations and tensor manipulation.
- b) OpenCV – Image and video processing.
- c) Matplotlib – Visualization of performance metrics and results.
- d) Pandas – Data handling and preprocessing.
- e) Scikit-learn – Performance evaluation utilities.

### 4. Additional Tools

- a) Git – Version control and project management.
- b) TensorBoard – Training visualization and monitoring.
- c) LabelImg / Roboflow – Annotation and dataset management.
- d) Jupyter Notebook / VS Code – Development environment.

## 3.4 SYSTEM MODULES

### 3.4.1 Dataset Preparation Module

This module is responsible for preparing the dataset for training and validation. It ensures that the input data is formatted and structured correctly for the YOLOv8 model.

Functions:

1. Load KITTI dataset images and annotations
2. Resize images to required dimensions (e.g., 640×640)
3. Normalize pixel values
4. Convert annotations into YOLO format
5. Split dataset into training and validation sets

Output:

Preprocessed dataset ready for YOLOv8 training.

### **3.4.2 YoloV8 Training Module**

This module handles the initialization and training of the YOLOv8 model for pedestrian detection. A pretrained YOLOv8 model is fine-tuned using the prepared dataset.

Functions:

- Select appropriate YOLOv8 variant (n/s/m/l/x)
- Configure hyperparameters (epochs, batch size, learning rate)
- Perform training using PyTorch backend
- Save best-performing model weights (.pt file)

Output:

Trained YOLOv8 model optimized for pedestrian detection.

### **3.4.3 Model Evaluation Module**

This module evaluates the performance of the trained model using standard object detection metrics to ensure reliability and accuracy.

Functions:

1. Compute Precision
2. Compute Recall
3. Calculate Mean Average Precision (mAP@50, mAP@50–95)
4. Measure inference speed (FPS)
5. Analyze detection results

Output:

Performance metrics and evaluation report.

### **3.4.4 Model Conversion Module (Onnx Export)**

This module converts the trained YOLOv8 model into an interoperable format suitable for optimization and deployment.

Functions:

1. Export YOLOv8 model (.pt) to ONNX format (.onnx)
2. Validate exported model compatibility

Output:

ONNX model ready for OpenVINO optimization.

### **3.4.5 Openvino Optimization Module**

This module optimizes the ONNX model for efficient CPU-based inference using the OpenVINO toolkit.

Functions:

1. Convert ONNX model to OpenVINO Intermediate Representation (IR) format
2. Apply graph-level optimizations
3. Reduce latency and improve inference speed

Output:

Optimized IR model (.xml and .bin files).

### **3.4.6 Inference Module**

This module performs real-time pedestrian detection using the optimized OpenVINO model.

Functions:

1. Load OpenVINO IR model
2. Accept image, video, or live camera input Perform
3. CPU-based inference
4. Generate bounding boxes and confidence scores

Output:

Detected pedestrian coordinates and confidence values

### **3.4.7 Visualization Module**

This module displays the detection results to the user in a visual format.

Functions:

1. Draw bounding boxes on detected pedestrians
2. Display confidence scores
3. Show Frames Per Second (FPS)

Output: Final visual output with real-time detection results.

## CHAPTER – 4

### SYSTEM DESIGN

#### 4.1 ARCHITECTURE

The architecture of the YOLOv8-based pedestrian detection system is modular, scalable, and optimized for rapid processing. It integrates data acquisition, preprocessing, model inference, visualization, and output layers into a streamlined pipeline.

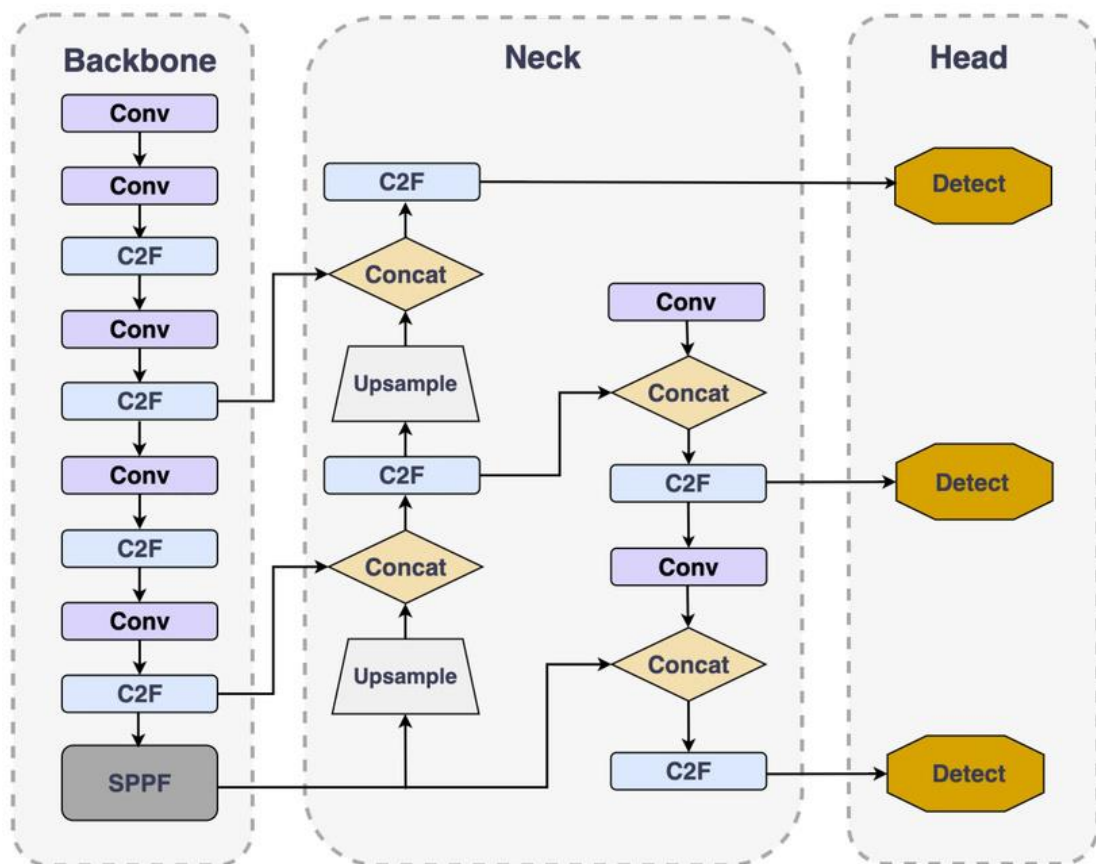


Figure 4.1 Model architecture of YOLO version-8

#### 4.2 DETAILED COMPONENTS

This subsection breaks down each architectural module, explaining its purpose, internal operations, and interactions with other components.

#### **4.2.1 Backbone Component**

The backbone is responsible for extracting important features from the input image. It consists of multiple Conv (Convolutional) layers and C2F blocks, which help in learning spatial and semantic information. The convolution layers capture low-level features such as edges and textures, while deeper layers capture high-level features like object shapes. The SPPF (Spatial Pyramid Pooling Fast) module at the end improves the receptive field and helps the model detect objects at different scales. Overall, the backbone converts the input image into meaningful feature maps.

#### **4.2.2 Neck Component**

The neck combines and refines features extracted from different stages of the backbone. It uses operations such as Up-sample, concat (Concatenation), and C2F blocks to merge features from multiple scales. Up-sampling increases the resolution of feature maps, while concatenation combines information from different layers. This helps the model detect both small and large pedestrians effectively. The neck essentially improves feature representation by enabling multi-scale feature fusion.

#### **4.2.3 Head Component**

The head is the final stage where actual object detection takes place. It receives processed feature maps from the neck and predicts bounding boxes, class labels, and confidence scores. The “Detect” blocks indicate detection at different scales, allowing the model to identify pedestrians of varying sizes in the image. This multi-scale detection capability is essential for real-time applications.

### **4.3 WORKFLOW**

The workflow of the proposed system begins with input acquisition in the form of images or video streams. The input data is then preprocessed and passed to the YOLOv8 model for pedestrian detection, where objects are identified using bounding boxes and confidence scores. The detected results are further optimized using OpenVINO to improve inference speed and performance. Finally, the output is displayed in real-time with accurate visualization of detected pedestrians.

In addition to the core detection pipeline, the system incorporates a structured training and evaluation process to ensure reliable performance. The model is trained using a well-annotated dataset, where data preprocessing techniques such as resizing, normalization, and augmentation are applied to enhance generalization. During evaluation, performance metrics such as precision, recall, mAP, and FPS are used to assess the effectiveness of the model. This ensures that the system maintains a balance between detection accuracy and real-time processing capability.

#### **Pedestrian Detection System Flow**

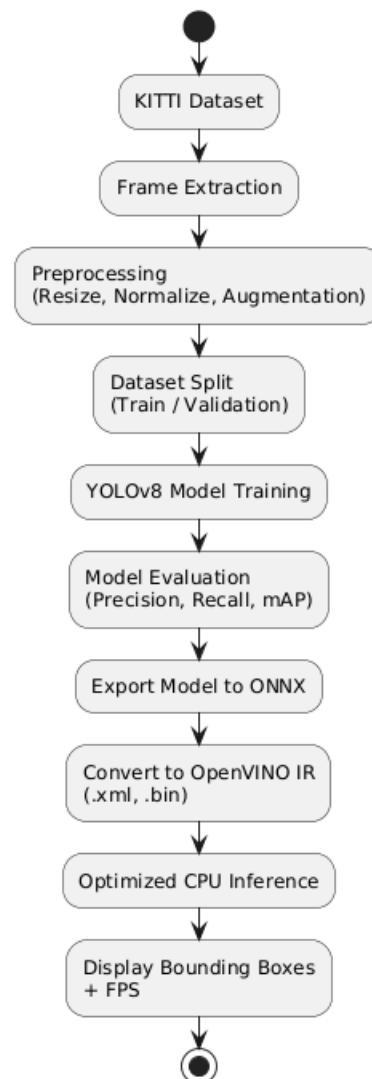


Figure 4.3 Data workflow diagram



## 4.4 UML DIAGRAMS

### 4.4.1 Use Case Diagram

The use case diagram illustrates the interaction between the user and the system. The user performs actions such as preparing the dataset, training the YOLOv8 model, and running real-time detection. The system includes additional functionalities like evaluating model performance and optimizing the model using OpenVINO. The relationships between use cases show dependencies such as optimization requiring model conversion. This diagram highlights how users interact with the system to achieve pedestrian detection.

The use case diagram further emphasizes the sequential flow of operations within the system, where each activity builds upon the previous one. For instance, dataset preparation is a foundational step that directly influences the quality of model training. Proper annotation, class filtering (focusing only on pedestrians), and dataset splitting into training and validation sets ensure that the YOLOv8 model learns effectively. Once the dataset is prepared, the training process utilizes this structured data to generate a robust detection model. This clearly reflects a dependency relationship in the diagram, where successful completion of earlier use cases is essential for the execution of subsequent ones.

In addition, the diagram captures the system's evaluation and validation phase, which plays a critical role in determining the effectiveness of the trained model. After training, the user can invoke performance evaluation to analyze metrics such as precision, recall, and mean Average Precision (mAP). These evaluation steps allow the user to assess whether the model meets the desired accuracy and real-time detection requirements. If the performance is not satisfactory, the diagram implicitly supports iterative refinement, where the user may return to earlier stages like dataset preparation or retraining, demonstrating a cyclic interaction rather than a strictly linear process.

Furthermore, the inclusion of model optimization using OpenVINO highlights the system's focus on deployment efficiency and real-world applicability. This use case depends on prior model conversion and is essential for improving inference speed, especially on CPU-based systems without dedicated GPUs. The optimized model enables faster real-time pedestrian detection, making the system suitable for practical applications such as surveillance or traffic monitoring. Overall, the use case diagram

not only represents functional interactions but also showcases system scalability, performance enhancement, and the integration of advanced optimization techniques to meet real-time operational demands.

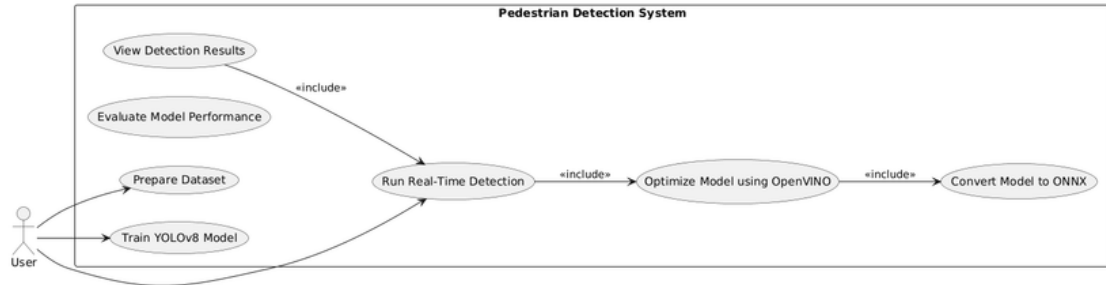


Figure 4.4.2 Use Case Diagram of Pedestrian Detection

#### 4.4.2 Class Diagram

The class diagram represents the object-oriented structure of the system and its key classes. It includes classes such as Dataset Manager, YOLO Trainer, OpenVINO Optimizer, Inference Engine, and Visualization. Each class contains specific methods like loading data, training the model, converting formats, running inference, and displaying results. The relationships between classes show how data flows from one component to another. This design ensures better organization, reusability, and maintainability of the code. Each class is designed with a specific responsibility, following modular programming principles, which simplifies debugging and future enhancements. The interaction between classes ensures a smooth flow of data throughout the system pipeline. This structured approach also allows easy integration of additional features or components without affecting existing functionality. Furthermore, it promotes code scalability, making the system adaptable for larger datasets and more complex detection tasks. Overall, the class diagram provides a clear blueprint for the implementation and management of the system.

The class diagram also illustrates the encapsulation of functionalities within each component, ensuring that internal processes are hidden while only essential methods are exposed. For example, the Dataset Manager class encapsulates operations such as data loading, preprocessing, annotation filtering, and dataset splitting, while the YOLO Trainer focuses solely on model initialization, training, and validation. Similarly, the OpenVINO Optimizer class handles model conversion and quantization processes independently. This separation of concerns reduces code complexity and

allows developers to modify or upgrade individual components without impacting the entire system.

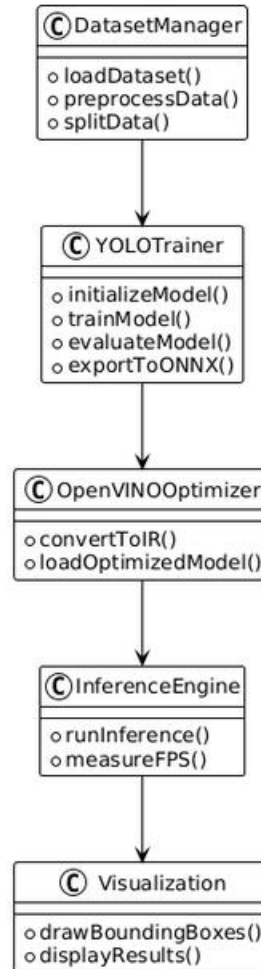


Figure 4.4.2 Class Diagram of Pedestrian Detection

#### 4.4.3 Sequence Diagram

The sequence diagram shows the chronological interaction between system components during execution. It starts with the user initiating model training using YOLOv8, followed by exporting the trained model to ONNX format. The ONNX model is then converted into OpenVINO IR format for optimization. The optimized model is used for inference, where bounding boxes and FPS are generated. Finally, the results are displayed to the user. This diagram clearly explains the step-by-step communication between different modules over time.

The sequence diagram further highlights the time-ordered flow of messages between components, ensuring that each process is executed in a well-defined sequence. It begins with the user providing input commands, which are handled by the system to initiate dataset loading and preprocessing. Once the data is prepared, the control is passed to the YOLOv8 training module, where the model undergoes multiple epochs of learning. During this phase, intermediate outputs such as loss values and accuracy metrics may also be generated, indicating the progress of training. This structured progression ensures that each step is completed before moving to the next, avoiding conflicts or incomplete operations.

Additionally, the diagram captures the transformation of the trained model across different formats, which is crucial for deployment. After training, the model is exported to ONNX format, acting as an intermediate representation that ensures compatibility across frameworks.

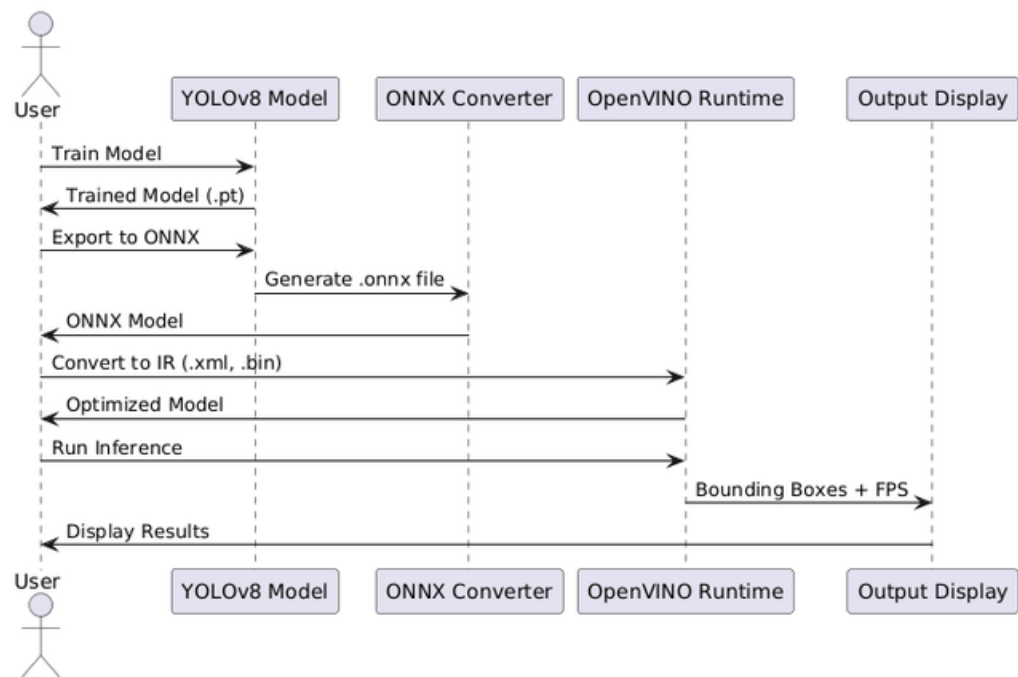


Figure 4.4.3 Sequence Diagram of Pedestrian Detection

#### 4.4.4 Activity Diagram

The activity diagram describes the step-by-step flow of operations in the system. It starts with dataset preparation and model training, followed by performance evaluation. A decision point checks whether the model performance is satisfactory. If

not, hyperparameters are tuned and the model is retrained. If satisfactory, the model is exported to ONNX and converted to OpenVINO format.

The activity diagram continues by illustrating the iterative nature of the system, particularly during the training and optimization phases. After the initial model training and evaluation, the decision node plays a crucial role in determining the next course of action. If the model does not meet the desired performance metrics such as accuracy, precision, or mAP, the workflow loops back to hyperparameter tuning. This may involve adjusting parameters like learning rate, batch size, or number of epochs to improve model performance. The retraining process ensures that the system gradually converges toward an optimal model through continuous refinement.

Once the model achieves satisfactory performance, the workflow proceeds to the deployment preparation stage. In this phase, the trained model is exported into ONNX format, which serves as a bridge between different frameworks.

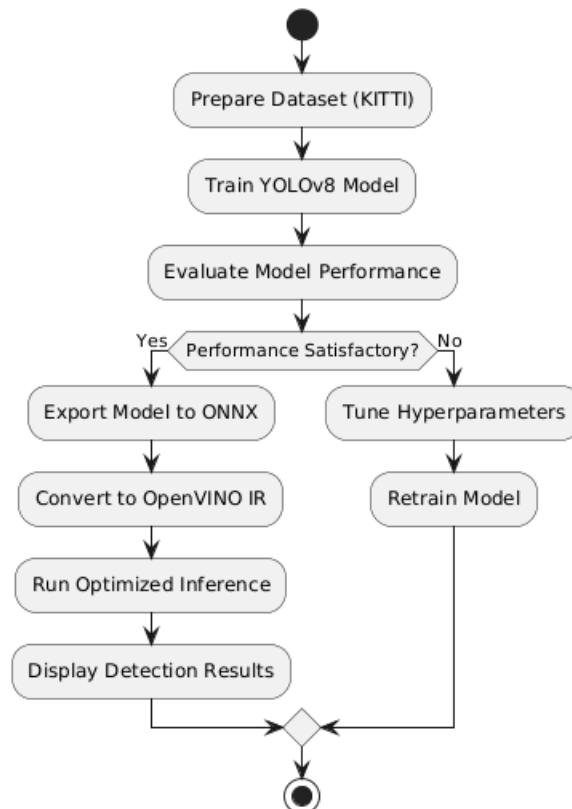


Figure 4.4.3 Activity Diagram of Pedestrian Detection

#### 4.4.5 Component Diagram

The component diagram shows the modular structure of the pedestrian detection system. It includes different modules such as the dataset module, YOLOv8 training module, evaluation module, and model conversion module. These modules interact with each other to process data and train the model efficiently. The OpenVINO optimization module improves model performance, and the inference module handles real-time detection. The visualization module displays the final results. This modular approach improves system scalability, maintainability, and clarity of design.

The component diagram further elaborates how each module operates as an independent unit while still contributing to the overall system functionality.

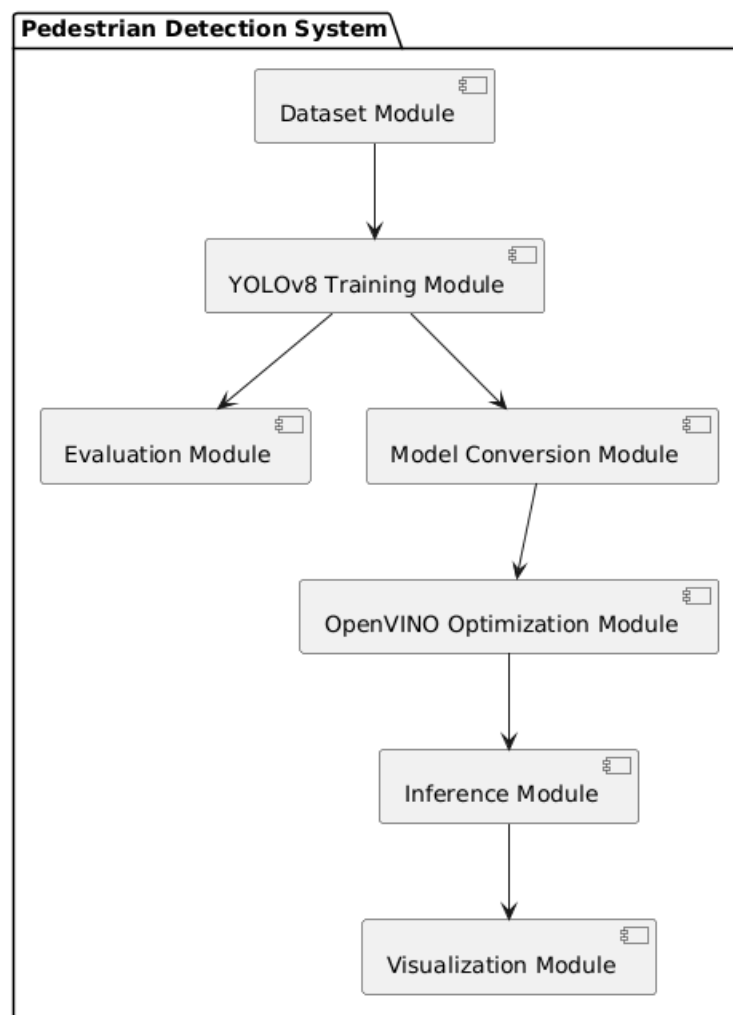


Figure 4.4.5 Component Diagram of Pedestrian Detection

# **CHAPTER – 5**

## **SYSTEM IMPLEMENTATION**

### **5.1 TECHONOLOGIES**

The development of a robust and efficient pedestrian detection system using YOLOv8 relies on a combination of modern programming technologies, deep learning frameworks, computer vision libraries, development tools, and hardware accelerators. These technologies work together to ensure the system achieves high accuracy, real time processing capability, and ease of deployment. This section provides a detailed analysis of each technology used, its purpose, and its relevance to the project.

#### **5.1.1 Programming Language and Environment**

##### **1. Python**

Python serves as the primary programming language for the project due to its simplicity, extensive library support, and strong integration with machine learning and computer vision frameworks.

Key Features:

- a) Easy-to-read syntax suitable for rapid prototyping
- b) Massive ecosystem of scientific and deep learning libraries
- c) Strong community support and continuous development
- d) Cross-platform compatibility

Python's flexibility makes it ideal for integrating deep learning workflows, performing dataset processing, writing inference scripts, and deploying the YOLOv8 model effectively.

#### **5.1.2 Deep Learning Frameworks**

##### **1. PyTorch**

PyTorch is an open-source deep learning framework developed by Facebook AI Research. It provides dynamic computation graphs, making model development and debugging more intuitive.

Why PyTorch was chosen:

- a) Dynamic graph execution enables flexible experimentation

- b) Native CUDA support for GPU acceleration
- c) Easy integration with YOLOv8 and Ultralytics libraries
- d) Strong developer community and clear documentation
- e) Highly efficient during both training and deployment

## 2. Ultralytics

**YOLOv8 Framework** YOLOv8 is the latest addition to the YOLO family and is developed by Ultralytics. It offers a complete pipeline for training, validation, testing, and deployment with minimal coding effort.

Key Features:

- a) Pretrained models available for transfer learning
- b) Highly optimized architecture for small-object detection
- c) Scalable model variations (YOLOv8n, s, m, l, x)
- d) Built-in data augmentation and preprocessing
- e) Decoupled detection head for improved accuracy
- f) Integrated CLI and Python API support

### 5.1.3 Computer Vision Libraries

Computer vision libraries play a major role in reading, processing, and visualizing image or video data for both training and testing the system.

#### 1. OpenCV (Open-Source Computer Vision Library)

OpenCV is an industry-standard library widely used for image processing and computer vision tasks.

Major capabilities used in this project:

- a) Reading images and video streams
- b) Preprocessing frames (resizing, normalization, color conversion)
- c) Drawing bounding boxes and labels on detected pedestrians
- d) Handling real-time video input from webcams or CCTV feeds
- e) Saving processed frames or annotated videos

OpenCV's efficiency and compatibility with Python make it an essential component of this project.



## 2. NumPy

NumPy is the numerical computing library used throughout the project for:

- a) Multi-dimensional array manipulation
- b) Normalizing image pixel values
- c) Performing matrix operations required during preprocessing
- d) Supporting transformations and tensor conversions

Deep learning models rely heavily on tensor operations, and NumPy provides the computational foundation.

## 3. Matplotlib

Matplotlib is used for:

- a) Plotting training accuracy and loss curves
- b) Visualizing model evaluation metrics
- c) Displaying sample detection outputs

These visualizations help analyze model behavior and performance trends over time.

### **5.1.4 Data Handling and Annotation Tools**

Preparing the dataset is a crucial step. Dataset annotation, conversion, and preprocessing tools ensure YOLOv8 can correctly interpret the training data.

#### 1. Roboflow

Roboflow is a cloud-based data management tool that simplifies dataset preparation.

Possible uses:

- a) Converting datasets to YOLOv8 format
- b) Applying preprocessing and augmentation
- c) Auto-splitting datasets into train/val/test

Roboflow enhances productivity, especially when handling large datasets like KITTI or COCO.

### **5.1.5 Machine Learning and Data Science Tools**

These tools help manage large datasets, evaluate model performance, and analyze metrics.

## 1. Pandas

Pandas is used for:

- a) Managing dataset metadata
- b) Generating statistics about annotation counts
- c) Analyzing detection results saved in CSV format

## 2. Scikit-learn

Scikit-learn supports:

- a) Train-test split (if not handled by YOLOv8)
- b) Evaluation metrics (precision, recall, F1-score)
- c) Performance analysis

While YOLOv8 already computes most metrics, Scikit-learn is used for additional evaluation if needed.

### **5.1.6 Version Control and Development Tools**

Managing code, experiments, and dependencies is essential for collaborative development.

## 1. Git and GitHub

Git is used for:

- a) Tracking code changes
- b) Maintaining consistent development history
- c) Collaborating with teammates

GitHub provides cloud-based repository hosting, issue tracking, and project management.

## 2. IDEs (Integrated Development Environments)

Common choices include:

- a) Visual Studio Code (VS Code)
- b) Jupyter Notebook
- c) PyCharm

These environments simplify development, debugging, visualization, and testing.

## 5.2 SOURCE CODE

### # model training

```
from google.colab import drive

drive.mount('/content/drive')

!pip install ultralytics opencv-python-headless --quiet


from ultralytics import YOLO

model=YOLO('/content/drive/MyDrive/pedestrian_detection/models/kitti_ped_v2_small/weights/best.pt')

print("Model loaded!")


import os

PROJECT_DIR = '/content/drive/MyDrive/pedestrian_detection'

folders = [

    'dataset/images/train',

    'dataset/images/val',

    'dataset/labels/train',

    'dataset/labels/val',

    'models',

    'results'

]

for folder in folders:

    full_path = os.path.join(PROJECT_DIR, folder)

    os.makedirs(full_path, exist_ok=True)

    print(f'Created: {full_path}')

import torch
```

```

import cv2

import numpy as np

from ultralytics import YOLO

import openvino as ov

print("PyTorch   :", torch.__version__)

print("CUDA available:", torch.cuda.is_available())

print("GPU name   :", torch.cuda.get_device_name(0) if torch.cuda.is_available() else
"N/A")

print("OpenCV    :", cv2.__version__)

print("OpenVINO   :", ov.__version__)

print("YOLOv8     : OK")


import os

for folder in ['kitti_majproj', 'kitti_majproj2']:

    path = f'/content/drive/MyDrive/{folder}'

    if os.path.exists(path):

        print(f'\nContents of {folder}:')

        for item in sorted(os.listdir(path)):

            print(f' {item}')

import zipfile

import os

print("Unzipping images... (this may take 3-5 minutes)")

with zipfile.ZipFile('/content/drive/MyDrive/kitti_majproj2/data_object_image_2.zip',
'r') as z:

    z.extractall('/content/kitti_raw')

print("Images done!")

print("Unzipping labels...")

```

```

with zipfile.ZipFile('/content/drive/MyDrive/kitti_majproj2/data_object_label_2.zip',
'r') as z:

    z.extractall('/content/kitti_raw')

print("Labels done!")

print("\nContents of /content/kitti_raw:")

for item in sorted(os.listdir('/content/kitti_raw')):

    print(f'{item}')

import os

print("Contents of training folder:")

for item in sorted(os.listdir('/content/kitti_raw/training')):

    print(f' {item}')

import os

import shutil

import random

from pathlib import Path

KITTI_IMAGES = '/content/kitti_raw/training/image_2'

KITTI_LABELS = '/content/kitti_raw/training/label_2'

OUTPUT_DIR = '/content/drive/MyDrive/pedestrian_detection/dataset'


IMG_W, IMG_H = 1242, 375

TRAIN_RATIO = 0.8

RANDOM_SEED = 42

KEEP_CLASSES = {'Pedestrian': 0}

def to_yolo(x1, y1, x2, y2):

    cx = ((x1 + x2) / 2) / IMG_W

    cy = ((y1 + y2) / 2) / IMG_H

```

```

w = (x2 - x1) / IMG_W

h = (y2 - y1) / IMG_H

return round(cx,6), round(cy,6), round(w,6), round(h,6)

def convert(label_path):

    lines = []

    for line in open(label_path):

        parts = line.strip().split()

        if len(parts) < 9 or parts[0] not in KEEP_CLASSES:

            continue

        x1,y1,x2,y2 = map(float, parts[4:8])

        cx,cy,w,h = to_yolo(x1,y1,x2,y2)

        lines.append(f'0 {cx} {cy} {w} {h}')

    return lines or None

random.seed(RANDOM_SEED)

label_files = sorted(Path(KITTI_LABELS).glob('*.txt'))

valid = []

skipped = 0

for lf in label_files:

    yolo_lines = convert(lf)

    img_path = lf.stem + '.png'

    img = Path(KITTI_IMAGES) / img_path

    if yolo_lines and img.exists():

        valid.append((lf, img, yolo_lines))

    else:

        skipped += 1

print(f'Total label files : {len(label_files)}')

```

```

print(f'With pedestrians : {len(valid)}')

print(f'Skipped : {skipped}')

random.shuffle(valid)

split = int(len(valid) * TRAIN_RATIO)

splits = {'train': valid[:split], 'val': valid[split:]}

for s, samples in splits.items():

    for lf, img, ylines in samples:

        shutil.copy2(img, f'{OUTPUT_DIR}/images/{s}/{img.name}')

        open(f'{OUTPUT_DIR}/labels/{s}/{lf.stem}.txt', 'w').write('\n'.join(ylines))

    print(f'{s}: {len(samples)} samples')

yaml_content = """path: /content/drive/MyDrive/pedestrian_detection/dataset

train: images/train

val: images/val

nc: 1

names:

    0: pedestrian

"""

yaml_path = f'{OUTPUT_DIR}/kitti_pedestrian.yaml'

open(yaml_path, 'w').write(yaml_content)

print(f'\nYAML saved to: {yaml_path}')

print("Dataset ready for training!")

from ultralytics import YOLO

model = YOLO('yolov8s.pt')

results = model.train(

    data = '/content/drive/MyDrive/pedestrian_detection/dataset/kitti_pedestrian.yaml',

    epochs = 100,

```

```

    patience = 15,

    imgsz    = 640,

    batch    = 8,

    device   = 0,

    iou       = 0.5,

    conf      = 0.001,

    hsv_h     = 0.015,

    hsv_s     = 0.7,

    hsv_v     = 0.4,

    flipud    = 0.3,

    mosaic    = 1.0,

    scale     = 0.5,

    name      = 'kitti_ped_v2_small',

    project   = '/content/drive/MyDrive/pedestrian_detection/models',

    save      = True,

    plots     = True,

)

print("\nTraining complete!")

print(f'Best weights saved at: {results.save_dir}/weights/best.pt')

from IPython.display import Image

import os

results_path =
'/content/drive/MyDrive/pedestrian_detection/models/kitti_ped_v2_small'

print("Saved files:")

for item in sorted(os.listdir(results_path)):

    print(f' {item}')

```



## #HTML

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Pedestrian Detection System</title>
```

```
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
```

```
</head>
```

```
<body>
```

```
  <!-- Metrics Bar -->
```

```
  <div class="metrics-bar">
```

```
    <div class="metric-card">
```

```
      <p class="metric-label">Model</p>
```

```
      <p class="metric-value green">YOLOv8s</p>
```

```
    </div>
```

```
    <div class="metric-card">
```

```
      <p class="metric-label">mAP50</p>
```

```
      <p class="metric-value blue">0.842</p>
```

```
    </div>
```

```
    <div class="metric-card">
```

```
      <p class="metric-label">Precision</p>
```

```
      <p class="metric-value blue">0.887</p>
```

```
    </div>
```

```
  <div class="metric-card">
```

```

    <p class="metric-label">Recall</p>

    <p class="metric-value blue">0.733</p>

</div>

<div class="metric-card">

    <p class="metric-label">Speedup</p>

    <p class="metric-value amber">2.24x</p>

</div>

<div class="metric-card">

    <p class="metric-label">Optimization</p>

    <p class="metric-value green">INT8</p>

</div>

</div>

<!-- Tabs -->

<div class="tabs">

    <button class="tab active" onclick="switchTab('image')">Image Detection</button>

    <button class="tab" onclick="switchTab('video')">Video Detection</button>

    <button class="tab" onclick="switchTab('dataset')">Dataset Browser</button>

</div>

<!-- Image Tab -->

<div id="tab-image" class="tab-content active">

    <div class="two-col">

        <!-- Left: Upload -->

        <div class="panel">

            <p class="panel-label">Input Image</p>

            <div class="upload-zone" id="img-drop-zone"

```

```

        onclick="document.getElementById('img-input').click()"
        ondragover="event.preventDefault()"
        ondrop="handleImgDrop(event)">
<div class="upload-icon"></div>
<p>Click to upload or drag & drop</p>
<p class="upload-sub">PNG, JPG supported</p>
<input type="file" id="img-input" accept=".png,.jpg,.jpeg"
        onchange="handleImgUpload(event)" hidden>
</div>
<img id="img-preview" class="preview hidden" alt="preview">
<button class="btn-detect" id="btn-detect-img"
        onclick="detectImage()" disabled>
    Detect Pedestrians
</button>
</div>
<!-- Right: Result -->
<div class="panel">
    <p class="panel-label">Detection Result</p>
    <div class="result-zone" id="img-result-zone">
        <p class="placeholder">Result appears here</p>
    </div>
    <div class="stats-row">
        <div class="stat-box">
            <p class="stat-label">Pedestrians</p>
            <p class="stat-val green" id="stat-count">—</p>
        </div>

```

```

<div class="stat-box">

  <p class="stat-label">Inference</p>

  <p class="stat-val blue" id="stat-time">—</p>

</div>

<div class="stat-box">

  <p class="stat-label">Avg confidence</p>

  <p class="stat-val amber" id="stat-conf">—</p>

</div>

</div>

</div>

</div>

</div>

<!-- Video Tab -->

<div id="tab-video" class="tab-content">

  <div class="two-col">

    <!-- Left: Upload -->

    <div class="panel">

      <p class="panel-label">Input Video</p>

      <div class="upload-zone" id="vid-drop-zone"

        onclick="document.getElementById('vid-input').click()"

        ondragover="event.preventDefault()"

        ondrop="handleVidDrop(event)">

        <div class="upload-icon video"></div>

        <p>Click to upload video</p>

        <p class="upload-sub">MP4, AVI, MOV supported</p>

        <input type="file" id="vid-input" accept=".mp4,.avi,.mov"

```

```

        onchange="handleVidUpload(event)" hidden>
    </div>

    <video id="vid-preview" class="preview hidden" controls></video>

    <button class="btn-detect" id="btn-detect-vid"

        onclick="detectVideo()" disabled>

        Process Video

    </button>

    <div id="vid-progress" class="progress hidden">

        <div class="progress-bar">

            <div class="progress-fill" id="progress-fill"></div>

        </div>

        <p class="progress-text" id="progress-text">Processing...</p>

    </div>

</div>

<!-- Right: Result -->

<div class="panel">

    <p class="panel-label">Detection Result</p>

    <div class="result-zone" id="vid-result-zone">

        <p class="placeholder">Processed video appears here</p>

    </div>

    <div class="stats-row">

        <div class="stat-box">

            <p class="stat-label">Total frames</p>

            <p class="stat-val green" id="vid-frames">—</p>

        </div>

        <div class="stat-box">

```

```

        <p class="stat-label">Total detections</p>

        <p class="stat-val blue" id="vid-dets">—</p>

    </div>

    <div class="stat-box">

        <p class="stat-label">Avg FPS</p>

        <p class="stat-val amber" id="vid-fps">—</p>

    </div>

</div>

</div>

</div>

</div>

<!-- Dataset Tab -->

<div id="tab-dataset" class="tab-content">

    <div class="dataset-header">

        <p class="panel-label">KITTI Validation Dataset — Pick an image to detect</p>

        <input type="text" id="search-input"

            placeholder="Search by filename..."

            oninput="filterImages(this.value)">

    </div>

    <div class="image-grid" id="image-grid">

        <p class="placeholder">Loading dataset images...</p>

    </div>

    <!-- Selected image result -->

    <div id="dataset-result" class="hidden">

        <div class="two-col">

            <div class="panel">

```

```

    <p class="panel-label">Original</p>

    <img id="ds-original" class="preview" alt="original">

</div>

<div class="panel">

    <p class="panel-label">Detected</p>

    <img id="ds-detected" class="preview" alt="detected">

    <div class="stats-row">

        <div class="stat-box">

            <p class="stat-label">Pedestrians</p>

            <p class="stat-val green" id="ds-count">—</p>

        </div>

        <div class="stat-box">

            <p class="stat-label">Inference</p>

            <p class="stat-val blue" id="ds-time">—</p>

        </div>

        <div class="stat-box">

            <p class="stat-label">Avg confidence</p>

            <p class="stat-val amber" id="ds-conf">—</p>

        </div>

    </div>

</div>

</div>

</div>

</div>

<!-- Footer -->

<footer>

```

<span>Pedestrian Detection using YOLOv8 and OpenVINO Optimization</span>

<span>Final Year Major Project</span>

</footer>

<script src="{{ url\_for('static', filename='script.js') }}"></script>

</body>

</html>

## #CSS

\* { margin: 0; padding: 0; box-sizing: border-box; }

body {

background: #0f1117;

color: #e0e0e0;

font-family: 'Segoe UI', sans-serif;

min-height: 100vh;

}

/\* Header \*/

header {

background: #1a1d27;

padding: 14px 24px;

display: flex;

align-items: center;

justify-content: space-between;

border-bottom: 1px solid #2a2d3a;

}

.header-left {



```
display: flex;

align-items: center;

gap: 10px;
}

.status-dot {

width: 10px;

height: 10px;

border-radius: 50%;

background: #00d4aa;
}

h1 {

font-size: 16px;

font-weight: 600;

color: #fff;
}

.header-badges {

display: flex;

gap: 8px;
}

.badge {

font-size: 11px;

padding: 4px 10px;

border-radius: 20px;

font-weight: 500;
}
```

```

.badge.green { background: #1e3a2f; color: #00d4aa; border: 1px solid #00d4aa44; }

.badge.blue { background: #1a2840; color: #4da6ff; border: 1px solid #4da6ff44; }

.badge.amber { background: #2a2010; color: #f0a500; border: 1px solid #f0a50044; }

/* Metrics Bar */

.metrics-bar {

  display: flex;

  gap: 12px;

  padding: 16px 24px;

  background: #13151f;

  border-bottom: 1px solid #2a2d3a;

  overflow-x: auto;

}

.metric-card {

  background: #1a1d27;

  border: 1px solid #2a2d3a;

  border-radius: 8px;

  padding: 10px 16px;

  min-width: 100px;

  text-align: center;

}

.metric-label {

  font-size: 11px;

  color: #666;

  margin-bottom: 4px;

  text-transform: uppercase;

  letter-spacing: 0.5px;

```

```

}

.metric-value {
  font-size: 18px;
  font-weight: 600;
}

.metric-value.green { color: #00d4aa; }
.metric-value.blue { color: #4da6ff; }
.metric-value.amber { color: #f0a500; }

/* Tabs */

.tabs {
  display: flex;
  gap: 4px;
  padding: 16px 24px 0;
  background: #0f1117;
  border-bottom: 1px solid #2a2d3a;
}

.tab {
  background: transparent;
  border: none;
  color: #666;
  font-size: 14px;
  padding: 10px 20px;
  cursor: pointer;
  border-bottom: 2px solid transparent;
  transition: all 0.2s;
}

```

```

.tab:hover { color: #aaa; }

.tab.active {
    color: #00d4aa;
    border-bottom: 2px solid #00d4aa;
}

/* Tab content */

.tab-content { display: none; padding: 24px; }

.tab-content.active { display: block; }


/* Two column layout */

.two-col {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 20px;
}

/* Panel */

.panel {
    background: #1a1d27;
    border: 1px solid #2a2d3a;
    border-radius: 10px;
    padding: 16px;
}

.panel-label {
    font-size: 11px;
    color: #666;
    text-transform: uppercase;

```

```

    letter-spacing: 0.8px;

    margin-bottom: 12px;
}

/* Upload zone */

.upload-zone {
    background: #13151f;

    border: 2px dashed #2a2d3a;

    border-radius: 8px;

    padding: 32px;

    text-align: center;

    cursor: pointer;

    transition: border-color 0.2s;
}

/* Image grid */

.image-grid {
    display: grid;

    grid-template-columns: repeat(auto-fill, minmax(160px, 1fr));

    gap: 10px;

    max-height: 400px;

    overflow-y: auto;

    padding: 4px;
}

.grid-item {
    background: #1a1d27;

    border: 1px solid #2a2d3a;

    border-radius: 6px;

```

```

padding: 8px;

cursor: pointer;

transition: border-color 0.2s;

text-align: center;
}

.grid-item:hover { border-color: #00d4aa; }

.grid-item.selected { border-color: #00d4aa; background: #1e3a2f; }

.grid-item p {

font-size: 11px;

color: #666;

margin-top: 6px;
}

/* Download button */

.btn-download {

display: inline-block;

margin-top: 12px;

padding: 10px 20px;

background: #1a2840;

color: #4da6ff;

border: 1px solid #4da6ff44;

border-radius: 8px;

font-size: 13px;

font-weight: 500;

cursor: pointer;

text-decoration: none;

transition: background 0.2s;

```

```

}

.btn-download:hover { background: #1f3050; }

/* Loading spinner */

.spinner {

    width: 36px;

    height: 36px;

    border: 3px solid #2a2d3a;

    border-top-color: #00d4aa;

    border-radius: 50%;

    animation: spin 0.8s linear infinite;

    margin: 0 auto;

}

@keyframes spin { to { transform: rotate(360deg); } }

/* Footer */

footer {

    background: #1a1d27;

    border-top: 1px solid #2a2d3a;

    padding: 14px 24px;

    display: flex;

    justify-content: space-between;

    font-size: 12px;

    color: #444;

    margin-top: 24px;

}

/* Scrollbar */

::-webkit-scrollbar { width: 6px; height: 6px; }

```

```
::-webkit-scrollbar-track { background: #13151f; }

::-webkit-scrollbar-thumb { background: #2a2d3a; border-radius: 3px; }

::-webkit-scrollbar-thumb:hover { background: #3a3d4a; }
```

```
/* Responsive */
```

```
@media (max-width: 768px) {

  .two-col { grid-template-columns: 1fr; }

  .metrics-bar { flex-wrap: wrap; }

  .header-badges { display: none; }

}
```

## #Java Script

```
function switchTab(tab) {

  document.querySelectorAll('.tab').forEach(t => t.classList.remove('active'));

  document.querySelectorAll('.tab-content').forEach(c => c.classList.remove('active'));

  document.getElementById('tab-' + tab).classList.add('active');

  event.target.classList.add('active');

  if (tab === 'dataset') loadDatasetImages();

}

let imgFile = null;

function handleImgUpload(event) {

  imgFile = event.target.files[0];

  if (!imgFile) return;

  const reader = new FileReader();

  reader.onload = e => {

    const preview = document.getElementById('img-preview');

    preview.src = e.target.result;
```



```

    preview.classList.remove('hidden');

    document.getElementById('img-drop-zone').style.display = 'none';

    document.getElementById('btn-detect-img').disabled = false;

};

reader.readAsDataURL(imgFile);
}

function handleImgDrop(event) {

    event.preventDefault();

    const file = event.dataTransfer.files[0];

    if (file) {

        document.getElementById('img-input').files = event.dataTransfer.files;

        handleImgUpload({ target: { files: [file] } });

    }

}

function detectImage() {

    if (!imgFile) return;

    const btn = document.getElementById('btn-detect-img');

    btn.disabled = true;

    btn.textContent = 'Detecting...';

    document.getElementById('img-result-zone').innerHTML =

        '<div class="spinner"></div>';

    formData.append('file', imgFile);

    fetch('/detect_image', { method: 'POST', body: formData })

        .then(r => r.json())

        .then(data => {

            if (data.error) {

```

```

        alert(data.error);

        return;
    }

    document.getElementById('img-result-zone').innerHTML =

        ``;

    document.getElementById('stat-count').textContent = data.count;

    document.getElementById('stat-time').textContent = data.inference_ms + 'ms';

    document.getElementById('stat-conf').textContent = data.avg_conf + '%';

    })

    .catch(err => alert('Error: ' + err))

    .finally(() => {

        btn.disabled = false;

        btn.textContent = 'Detect Pedestrians';

    });

}

let vidFile = null;

function handleVidUpload(event) {

    vidFile = event.target.files[0];

    if (!vidFile) return;

    const preview = document.getElementById('vid-preview');

    preview.src = URL.createObjectURL(vidFile);

    preview.classList.remove('hidden');

    document.getElementById('vid-drop-zone').style.display = 'none';

    document.getElementById('btn-detect-vid').disabled = false;

}

```

```

function handleVidDrop(event) {
  event.preventDefault();

  const file = event.dataTransfer.files[0];

  if (file) {
    handleVidUpload({ target: { files: [file] } });
  }
}

function detectVideo() {
  if (!vidFile) return;

  const btn = document.getElementById('btn-detect-vid');
  btn.disabled = true;

  btn.textContent = 'Processing...';

  const progress = document.getElementById('vid-progress');
  const fill = document.getElementById('progress-fill');
  const progressTxt = document.getElementById('progress-text');
  progress.classList.remove('hidden');

  document.getElementById('vid-result-zone').innerHTML =
    '<div class="spinner"></div>';

  // Animate progress bar
  let pct = 0;

  const interval = setInterval(() => {
    pct = Math.min(pct + Math.random() * 3, 90);

    fill.style.width = pct + '%';

    progressTxt.textContent = `Processing video... ${Math.round(pct)}%`;

  }, 500);

  const formData = new FormData();

```

```

formData.append('file', vidFile);

fetch('/detect_video', { method: 'POST', body: formData })

.then(r => r.json())

.then(data => {

  clearInterval(interval);

  fill.style.width = '100%';

  progressTxt.textContent = 'Done!';

  if (data.error) {

    alert(data.error);

    return;

  }

  document.getElementById('vid-result-zone').innerHTML = `

<div style="padding:16px;text-align:center">

<p style="color:#00d4aa;font-size:14px;margin-bottom:12px">

  Video processed successfully!

</p>

<a href="/download_video/${data.output_video}"

  class="btn-download">

  Download Detected Video

</a>

</div>`;

  document.getElementById('vid-frames').textContent = data.total_frames;

  document.getElementById('vid-dets').textContent = data.total_dets;

  document.getElementById('vid-fps').textContent = data.avg_fps;

  })

.catch(err => {

```

```

        clearInterval(interval);

        alert('Error: ' + err);
    })

    .finally(() => {

        btn.disabled = false;

        btn.textContent = 'Process Video';

    });
}

```

```

let allImages = [];

function loadDatasetImages() {

    const grid = document.getElementById('image-grid');

    grid.innerHTML = '<p class="placeholder">Loading...</p>';

    fetch('/dataset_images')

        .then(r => r.json())

        .then(data => {

            allImages = data.images;

            renderGrid(allImages);

        })

        .catch(() => {

            grid.innerHTML = '<p class="placeholder">Failed to load images</p>';

        });

}

function renderGrid(images) {

    const grid = document.getElementById('image-grid');

    if (images.length === 0) {

```

```

    grid.innerHTML = '<p class="placeholder">No images found</p>';

    return;

}

grid.innerHTML = images.map(name => `

    <div class="grid-item" onclick="selectDatasetImage('${name}', this)">

        <p>${name}</p>

    </div>

`).join("");
}

```

```

function filterImages(query) {

    const filtered = allImages.filter(name =>

        name.toLowerCase().includes(query.toLowerCase())

    );

    renderGrid(filtered);

}

```

```

function selectDatasetImage(filename, el) {

    document.querySelectorAll('.grid-item').forEach(

        g => g.classList.remove('selected')

    );

    el.classList.add('selected');


    document.getElementById('dataset-result').classList.remove('hidden');

    document.getElementById('ds-original').src = "";

    document.getElementById('ds-detected').src = "";

```

```

document.getElementById('ds-count').textContent = '—';
document.getElementById('ds-time').textContent = '—';
document.getElementById('ds-conf').textContent = '—';

document.getElementById('ds-detected').style.display = 'none';
const spinner = document.createElement('div');
spinner.className = 'spinner';
spinner.id = 'ds-spinner';
document.getElementById('ds-detected').after(spinner);
fetch('/detect_image', {
  method : 'POST',
  headers: { 'Content-Type': 'application/json' },
  body   : JSON.stringify({ filename })
})
.then(r => r.json())
.then(data => {
  const sp = document.getElementById('ds-spinner');
  if (sp) sp.remove();

  if (data.error) {
    alert(data.error);
    return;
  }

  const orig = document.getElementById('ds-original');
  const det = document.getElementById('ds-detected');
  orig.src = 'data:image/jpeg;base64,' + data.original;

```

```

det.src = 'data:image/jpeg;base64,' + data.detected;

det.style.display = 'block';


document.getElementById('ds-count').textContent = data.count;

document.getElementById('ds-time').textContent = data.inference_ms + 'ms';

document.getElementById('ds-conf').textContent = data.avg_conf + '%';

}))

.catch(err => {

    const sp = document.getElementById('ds-spinner');

    if (sp) sp.remove();

    alert('Error: ' + err);

});

}

```

## **#Flask**

```

from flask import Flask, render_template, request, jsonify, send_file

import cv2

import numpy as np

import opencv as ov

import os

import time

import base64

from pathlib import Path

from werkzeug.utils import secure_filename

app = Flask(__name__)

```



```

OV_XML = r'G:\My
Drive\pedestrian_detection\models\openvino_int8\best_int8_openvino_model\best.x
ml'

VAL_IMGS = r'G:\My Drive\pedestrian_detection\dataset\images\val'

UPLOAD_DIR = r'G:\My Drive\pedestrian_detection\webapp\uploads'

RESULTS_DIR = r'G:\My Drive\pedestrian_detection\webapp\results'

CONF_THRES = 0.25

IOU_THRES = 0.45

IMG_SIZE = 640

ALLOWED_IMG = {'png', 'jpg', 'jpeg'}

ALLOWED_VID = {'mp4', 'avi', 'mov'}

# Load OpenVINO model once at startup

print("Loading OpenVINO INT8 model...")

core = ov.Core()

model = core.read_model(OV_XML)

compiled = core.compile_model(

    model = model,

    device_name = 'CPU',

    config = {

        'PERFORMANCE_HINT' : 'LATENCY',

        'NUM_STREAMS' : '1',

        'INFERENCE_NUM_THREADS' : '4',

    }

)

req = compiled.create_infer_request()

print("Model loaded!")

```

```

def allowed_file(filename, allowed):

    return '.' in filename and filename.rsplit('.', 1)[1].lower() in allowed

def detect(img):

    orig_h, orig_w = img.shape[:2]

    padded, r, pl, pt = letterbox(img, IMG_SIZE)

    inp = padded[:, :, :-1].transpose(2, 0, 1)[np.newaxis, :].astype(np.float32)/255.0

    out = req.infer({0: inp})[compiled.output(0)][0].T

    boxes = []

    for p in out:

        if p[4] < CONF_THRES:

            continue

        cx, cy, w, h = p[:4]

        x1 = (cx-w/2-pl)/r; y1 = (cy-h/2-pt)/r

        x2 = (cx+w/2-pl)/r; y2 = (cy+h/2-pt)/r

        x1, y1 = max(0, x1), max(0, y1)

        x2, y2 = min(orig_w, x2), min(orig_h, y2)

        if (y2-y1) > (x2-x1):

            boxes.append([x1, y1, x2, y2, float(p[4])])

    boxes = sorted(boxes, key=lambda x: x[4], reverse=True)

    keep = []

    while boxes:

        best = boxes.pop(0)

        keep.append(best)

        boxes = [b for b in boxes if

            (min(best[2], b[2]) - max(best[0], b[0])) *

            (min(best[3], b[3]) - max(best[1], b[1])) /

```

```

        ((best[2]-best[0])*(best[3]-best[1]) +
         (b[2]-b[0])*(b[3]-b[1]) + 1e-6) < IOU_THRES]

    return keep

def draw_boxes(img, boxes):

    out = img.copy()

    for box in boxes:

        x1,y1,x2,y2,conf = box

        x1,y1,x2,y2 = int(x1),int(y1),int(x2),int(y2)

        cv2.rectangle(out, (x1,y1), (x2,y2), (0,255,0), 2)

        label = f'Pedestrian {conf:.2f}'

        (lw,lh),_ = cv2.getTextSize(label,

                                     cv2.FONT_HERSHEY_SIMPLEX, 0.6, 2)

        cv2.rectangle(out, (x1,y1-25),(x1+lw+8,y1),(0,255,0),-1)

        cv2.putText(out, label, (x1+4,y1-8),

                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,0,0), 2)

    return out

def img_to_base64(img):

    _, buf = cv2.imencode('.jpg', img)

    return base64.b64encode(buf).decode('utf-8')

@app.route('/')

def index():

    return render_template('index.html')

@app.route('/dataset_images')

def dataset_images():

    images = sorted([f for f in os.listdir(VAL_IMGS)

                     if f.endswith('.png') or f.endswith('.jpg')])

```

```

    return jsonify({'images': images[:50]}) # first 5

@app.route('/detect_image', methods=['POST'])
def detect_image():
    if 'file' not in request.files and 'filename' not in request.json:
        return jsonify({'error': 'No file provided'}), 400
    if 'file' in request.files:
        file = request.files['file']
        if not allowed_file(file.filename, ALLOWED_IMG):
            return jsonify({'error': 'Invalid file type'}), 400
        fname = secure_filename(file.filename)
        path = os.path.join(UPLOAD_DIR, fname)
        file.save(path)
        img = cv2.imread(path)
    else:
        fname = request.json['filename']
        img = cv2.imread(os.path.join(VAL_IMGS, fname))
    if img is None:
        return jsonify({'error': 'Cannot read image'}), 400
    start = time.time()
    boxes = detect(img)
    elapsed= (time.time()-start)*1000
    result = draw_boxes(img, boxes)
    confs = [round(b[4]*100, 1) for b in boxes]
    avg_conf = round(sum(confs)/len(confs), 1) if confs else 0
    return jsonify({
        'original' : img_to_base64(img),

```

```

        'detected' : img_to_base64(result),

        'count'    : len(boxes),

        'inference_ms': round(elapsed, 1),

        'avg_conf'  : avg_conf,

        'confidences': confs,

    })

@app.route('/detect_video', methods=['POST'])
def detect_video():

    if 'file' not in request.files:

        return jsonify({'error': 'No file provided'}), 400

    file = request.files['file']

    if not allowed_file(file.filename, ALLOWED_VID):

        return jsonify({'error': 'Invalid file type'}), 400

    fname = secure_filename(file.filename)

    input_path = os.path.join(UPLOAD_DIR, fname)

    output_path = os.path.join(RESULTS_DIR, 'detected_' + fname)

    file.save(input_path)

    cap = cv2.VideoCapture(input_path)

    fps  = cap.get(cv2.CAP_PROP_FPS)

    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))

    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

    total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    fourcc = cv2.VideoWriter_fourcc(*'mp4v')

    out  = cv2.VideoWriter(output_path, fourcc, fps, (width, height))

    frame_count = 0

    total_dets = 0

```

```

total_time = 0

while True:

    ret, frame = cap.read()

    if not ret:

        break

    start = time.time()

    boxes = detect(frame)

    elapsed= (time.time()-start)*1000

    total_time += elapsed

    result = draw_boxes(frame, boxes)

    # Add overlay

    cv2.putText(result, f'Pedestrians: {len(boxes)}',

                (10,30), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0,255,255), 2)

    cv2.putText(result, f'OpenVINO INT8 | YOLOv8s',

                (10,60), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255,255,0), 2)

    out.write(result)

    total_dets += len(boxes)

    frame_count += 1

cap.release()

out.release()

avg_fps = round(1000 / (total_time/frame_count), 1) if frame_count else 0

return jsonify({

    'output_video' : 'detected_' + fname,

    'total_frames' : frame_count,

    'total_dets' : total_dets,

    'avg_dets' : round(total_dets/frame_count, 2) if frame_count else 0,

```

```

        'avg_fps' : avg_fps,
    })

@app.route('/download_video/<filename>')
def download_video(filename):
    path = os.path.join(RESULTS_DIR, filename)
    return send_file(path, as_attachment=True)

if __name__ == '__main__':
    app.run(debug=True, port=5000)

```

## 5.3 OUPUT SCREENSHOTS

### 5.3.1 Input Design

Input design focuses on how data is provided to the system for processing. In the proposed pedestrian detection system, the primary inputs are images and video streams obtained from cameras or stored datasets. These inputs may vary in resolution, lighting conditions, and environmental complexity. The system accepts input in common formats such as JPEG, PNG, and video formats like MP4. Before processing, the input data undergoes preprocessing steps such as resizing, normalization, and format conversion to match the requirements of the YOLOv8 model. Proper input design ensures that the data is consistent, noise-free, and suitable for accurate detection. Additionally, the system is designed to handle real-time input from webcams, enabling continuous monitoring and detection of pedestrians.

| Input Type | Description               |
|------------|---------------------------|
| Image      | Static image from dataset |
| Video      | Recorded video file       |

Table 5.3 Input Types

### 5.3.2 Output Design

Output design defines how the results of the system are presented to the user. The output of the proposed system consists of detected pedestrians displayed using bounding boxes along with confidence scores. These results are visualized in real-time on the screen, providing clear and interpretable information. The system highlights each detected pedestrian with a rectangular bounding box and labels it with a confidence value indicating the accuracy of detection. In addition to visual output, performance metrics such as Frames Per Second (FPS) may also be displayed to indicate system efficiency. The output is designed to be user-friendly and easy to understand, making it suitable for applications such as surveillance and traffic monitoring.



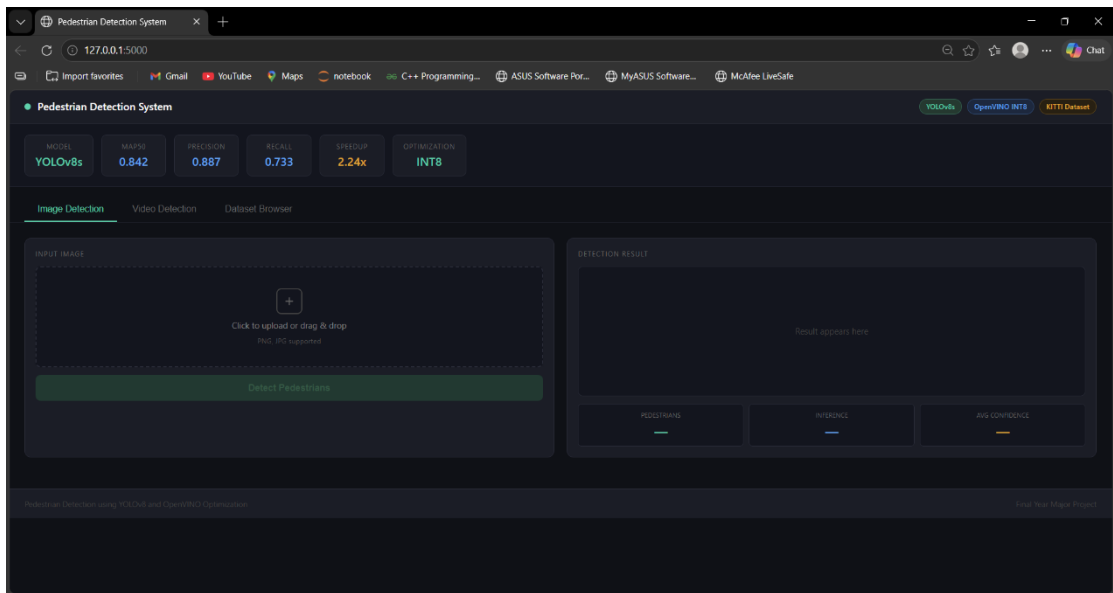


Figure 5.3.1 Image Detection Interface for Uploading and Detecting Pedestrians

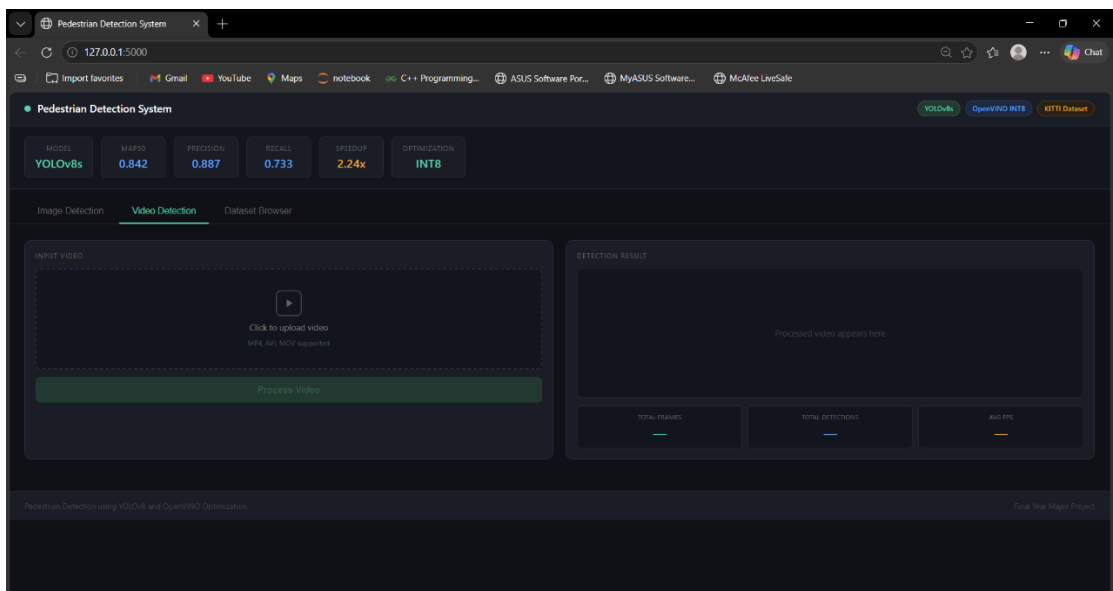


Figure 5.3.2 Video Detection Interface for Processing Pedestrian Detection in Videos

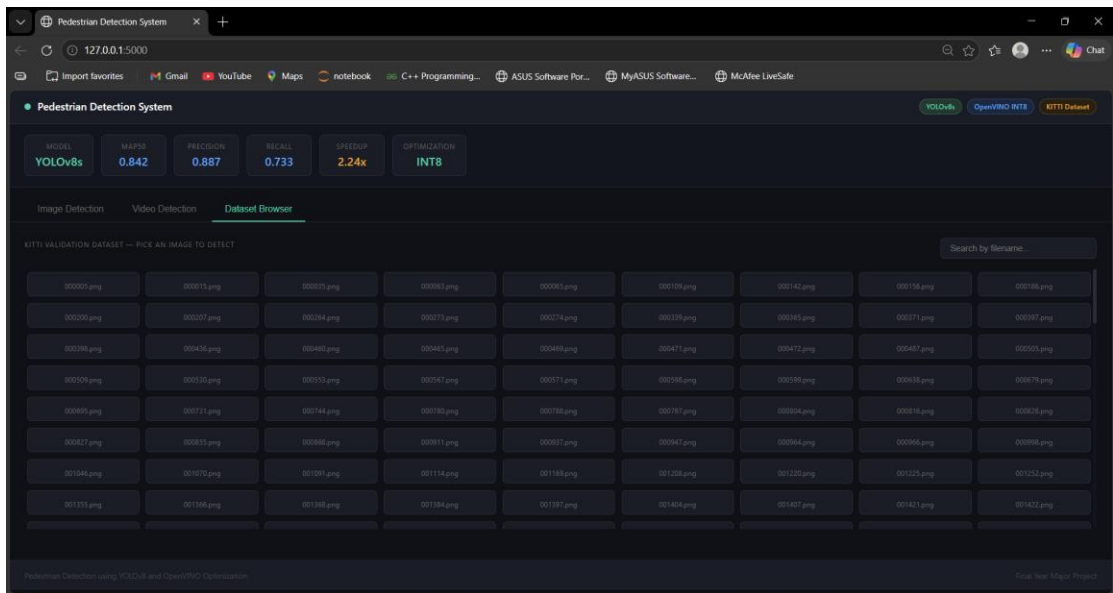


Figure 5.3.3 Dataset Browser Displaying KITT Validation Images

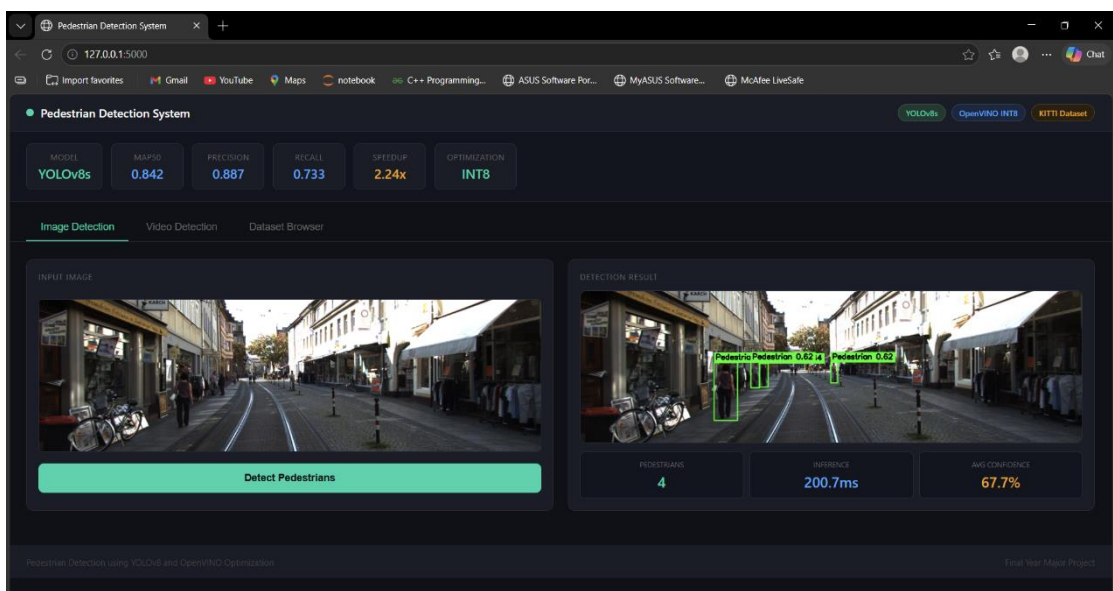


Figure 5.3.4 Pedestrian Detection Output with Bounding Boxes and Performance Metrics

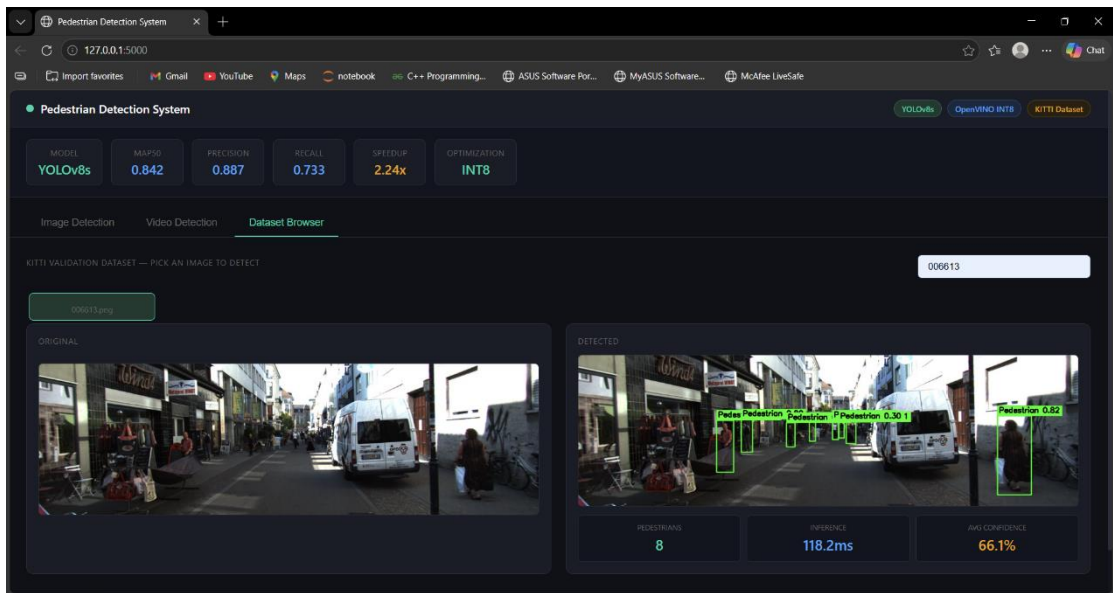


Figure 5.3.5 Dataset Browser Detection Result Showing Original and Detected Images

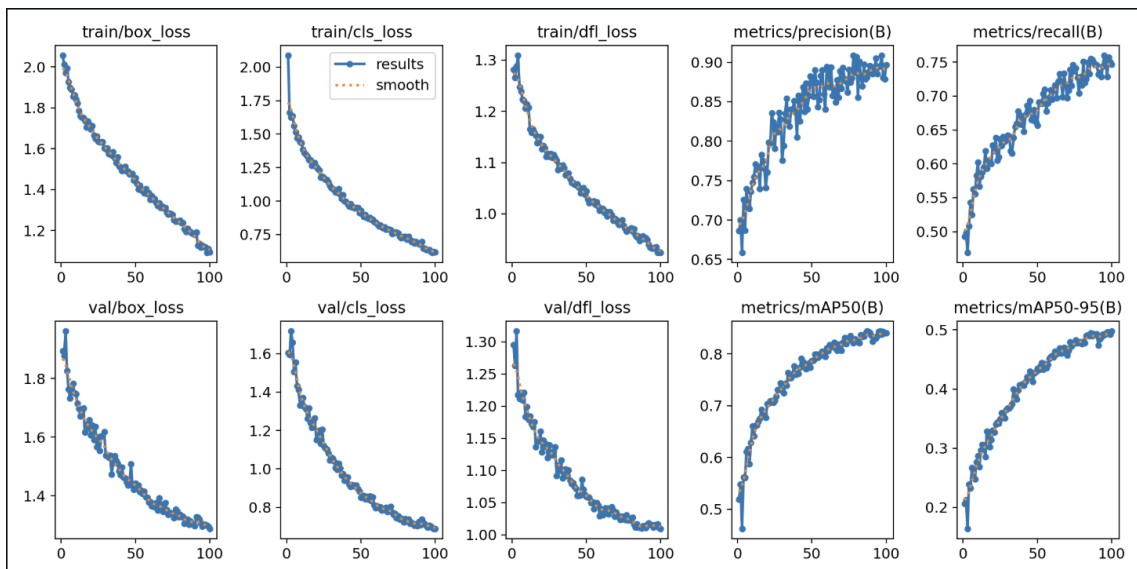


Figure 5.3.6 Training and Validation Performance Metrics of YOLOv8 Model

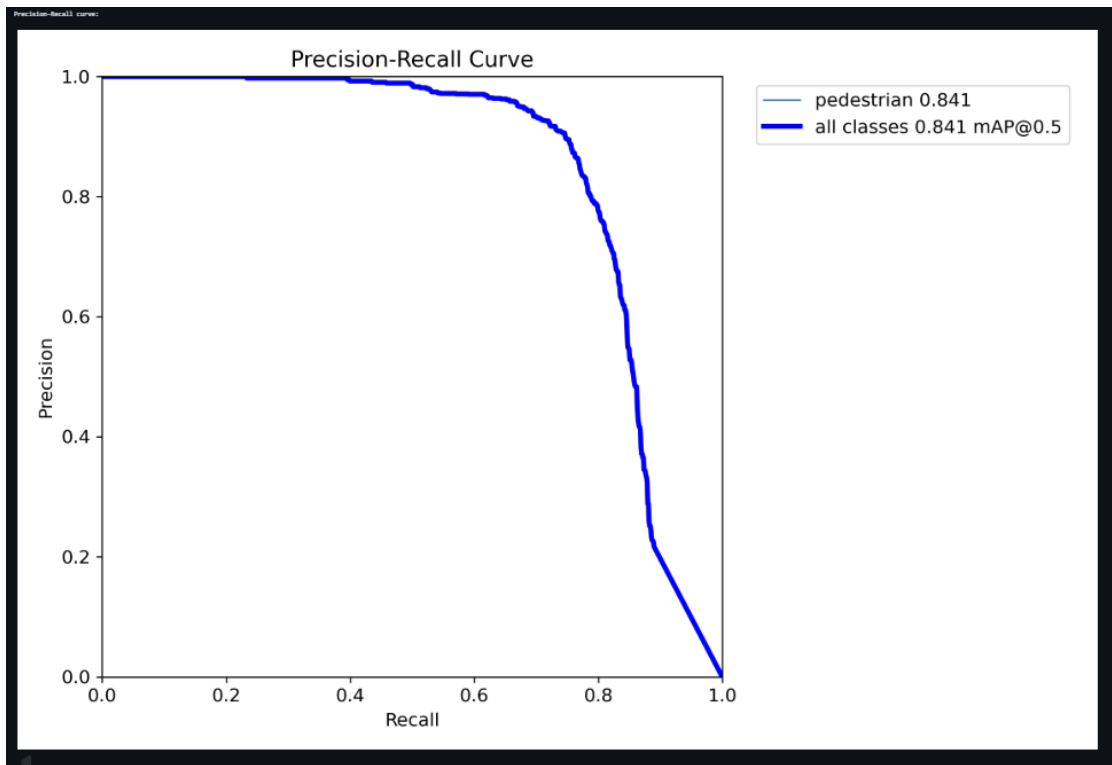


Figure 5.3.7 Precision–Recall Curve of YOLOv8 Model for Pedestrian Detection



Figure 5.3.8 Performance Comparison of YOLOv8 Using PyTorch and OpenVINO Optimization

## **CHAPTER – 6**

### **SYSTEM TESTING**

#### **6.1 TESTING OVERVIEW**

The testing and validation phase ensures that the proposed pedestrian detection system functions accurately, efficiently, and reliably under different conditions. Various testing techniques such as white box testing, black box testing, and performance testing are conducted to evaluate both the internal logic and external behavior of the system. The model is tested using diverse inputs, including images with single and multiple pedestrians, as well as challenging scenarios like low lighting and occlusions. The results demonstrate that the system performs effectively with high accuracy and real time speed, confirming its suitability for practical applications.

The testing and validation phase plays a crucial role in ensuring that the proposed pedestrian detection system operates accurately, efficiently, and reliably across a wide range of real-world scenarios. This phase is designed to verify not only the correctness of the system's outputs but also its robustness and stability under varying environmental and operational conditions. A structured testing approach is followed to identify potential errors, validate model performance, and ensure that all system components function as intended.

To achieve comprehensive validation, multiple testing techniques are employed, including white box testing, black box testing, unit testing, integration testing, and performance testing. White box testing focuses on verifying the internal logic and code structure of the system, ensuring that all conditions, loops, and execution paths are functioning correctly. Black box testing evaluates the system from a user's perspective by providing different input images and verifying whether the expected outputs, such as accurate pedestrian detection and bounding boxes, are generated. Additionally, unit testing ensures that individual modules such as data preprocessing, annotation conversion, and model inference work correctly, while integration testing verifies the smooth interaction between these modules.

The system is tested using a diverse dataset that includes images and video frames containing single as well as multiple pedestrians. Special attention is given to challenging scenarios such as low-light environments, partial occlusions, crowded

scenes, and varying camera angles. These conditions help evaluate the model's robustness and generalization capability. By exposing the system to such variations, the testing phase ensures that the model can maintain consistent detection accuracy even in complex real-world situations.

Furthermore, performance testing is conducted to measure key metrics such as inference time, frames per second (FPS), latency, and resource utilization on different hardware platforms, including edge devices and standard computing systems. The results demonstrate that the system achieves high detection accuracy while maintaining real-time performance, especially after optimization techniques such as quantization and hardware acceleration. Overall, the testing outcomes confirm that the proposed system is reliable, scalable, and suitable for deployment in practical applications such as surveillance, smart cities, and autonomous systems. To ensure the pedestrian detection system is accurate, reliable, and efficient, multiple testing techniques are applied. Each testing type focuses on a different aspect of system validation—from individual components to overall system performance. Below is a detailed explanation of each testing method along with structured tables for clear presentation in your report.

## **6.2 Unit Testing**

Unit testing is performed to validate individual modules of the system in isolation. The goal is to ensure that each component functions correctly before it is integrated with other modules. In this project, unit testing includes verifying data preprocessing functions, annotation conversion from KITTI to YOLO format, dataset loading mechanisms, and model inference functions. By testing these units independently, errors can be detected early, reducing complexity during later stages.

This testing helps improve code quality and reliability by ensuring that every small part of the system behaves as expected. It also simplifies debugging since issues can be traced back to a specific module.

| Component Tested         | Test Description                       | Expected Outcome                     | Result |
|--------------------------|----------------------------------------|--------------------------------------|--------|
| Data Preprocessing       | Check image resizing and normalization | Images correctly formatted for model | Pass   |
| Annotation Conversion    | Convert KITTI format to YOLO format    | Accurate bounding box conversion     | Pass   |
| Dataset Loading          | Load images and labels                 | Data loaded without errors           | Pass   |
| Model Inference Function | Run model on sample input              | Correct detection output             | Pass   |

Table 6.1 Unit Testing

### 6.3 Integration Testing

Integration testing focuses on verifying the interaction between different modules of the system. After individual components are tested, they are combined to check whether they work together seamlessly. In this system, integration testing ensures that the dataset module correctly feeds data into the training module, and the trained model integrates properly with evaluation and inference modules. This type of testing helps identify issues related to data flow, compatibility, and communication between modules, ensuring that the system operates as a cohesive unit. Additionally, integration testing also validates the end-to-end workflow of the system, ensuring that the complete pipeline—from data input to final output—functions without interruptions or inconsistencies. This includes verifying that preprocessed data is correctly consumed by the training module, trained models are accurately evaluated using validation datasets, and the optimized models are seamlessly deployed for inference. Special attention is given to handling errors such as data format mismatches, missing dependencies, or incorrect model configurations. By simulating real execution scenarios, integration testing ensures that all interconnected components operate reliably together, thereby improving system stability and reducing the risk of failures during deployment in real-world environments.

| Modules Involved               | Test Description                   | Expected Outcome                   | Result |
|--------------------------------|------------------------------------|------------------------------------|--------|
| Dataset + Training Module      | Feed dataset into YOLOv8 training  | Model trains without errors        | Pass   |
| Training + Evaluation Module   | Evaluate trained model             | Correct accuracy metrics generated | Pass   |
| Model + Inference Module       | Run trained model on input images  | Real-time detection works properly | Pass   |
| Conversion + Deployment Module | Convert model to OpenVINO/TensorRT | Optimized model runs successfully  | Pass   |

Table 6.2 Integration Testing

## 6.4 White Box Testing

White box testing is used to examine the internal structure and logic of the system. It involves analyzing the code to ensure that all execution paths, loops, and conditions are functioning correctly. Developers test internal workflows such as training scripts, preprocessing pipelines, and optimization processes. This testing improves code efficiency, ensures logical correctness, and helps detect hidden errors that may not be visible through external testing.

In addition, white box testing includes techniques such as control flow testing, data flow testing, and path coverage analysis to ensure that every possible execution path in the code is validated. For the pedestrian detection system, this means verifying that image preprocessing steps handle different input formats correctly, training loops update weights as expected, and loss calculations are accurate across epochs. Developers also check exception handling mechanisms to ensure the system behaves predictably under unexpected conditions, such as corrupted data or missing files. By thoroughly testing the internal logic, white box testing enhances system reliability, reduces runtime errors, and ensures that the overall implementation is robust and maintainable.



| Code Section           | Test Description                         | Expected Outcome              | Result |
|------------------------|------------------------------------------|-------------------------------|--------|
| Preprocessing Script   | Verify data transformation logic         | Correct data formatting       | Pass   |
| Training Pipeline      | Check training loop and loss calculation | Proper model convergence      | Pass   |
| Condition Handling     | Test edge cases and conditions           | No runtime errors             | Pass   |
| Model Conversion Logic | Validate OpenVINO conversion steps       | Successful model optimization | Pass   |

Table 6.3 White Box Testing

## 6.5 Black Box Testing

Black box testing evaluates the system based on its functionality without considering internal code structure. The system is tested using various input images and videos, and the outputs are compared against expected results. This includes testing with single and multiple pedestrians, crowded scenes, low-light conditions, and occlusions. This method ensures that the system meets user requirements and performs accurately in real-world scenarios.

In addition, black box testing also focuses on validating the overall user experience and system behavior under different practical conditions. This includes checking the accuracy of bounding box placement, confidence scores of detections, and consistency of results across different inputs. The system is also tested for robustness by providing unexpected or edge-case inputs, such as blurred images, partial visibility of pedestrians, and varying camera angles. Any discrepancies between expected and actual outputs are carefully analyzed to improve model performance. By emphasizing functionality and real-world applicability, black box testing ensures that the system delivers reliable and user-acceptable results without requiring knowledge of its internal implementation.

| Input Type              | Test Description                 | Expected Outcome                   | Result |
|-------------------------|----------------------------------|------------------------------------|--------|
| Single Pedestrian Image | Detect one person in image       | Accurate bounding box detection    | Pass   |
| Multiple Pedestrians    | Detect multiple persons          | All pedestrians detected correctly | Pass   |
| Low-Light Images        | Test detection in poor lighting  | Acceptable detection accuracy      | Pass   |
| Occluded Pedestrians    | Detect partially visible persons | Partial detection maintained       | Pass   |

Table 6.4 Black Box Testing

## 6.6 Performance Testing

Performance testing evaluates how efficiently the system operates under different conditions. It measures key metrics such as inference time, frames per second (FPS), latency, and resource utilization (CPU/GPU usage). The system is tested on different hardware platforms including edge devices and standard systems to ensure real-time performance. Optimization techniques such as quantization (INT8) and hardware acceleration (OpenVINO, TensorRT) are also evaluated to compare performance improvements. This ensures that the system is scalable and suitable for deployment in real-world applications.

Performance testing also involves analyzing the system's behavior under varying workloads and input sizes to assess its stability and consistency. The model is evaluated with high-resolution images, continuous video streams, and large datasets to observe how performance metrics change under stress conditions. Comparisons are made between different YOLOv8 model variants (nano to xlarge) to identify the best trade-off between accuracy and speed based on hardware capabilities. The testing process also helps identify bottlenecks such as memory limitations or processing delays, enabling further optimization. By thoroughly evaluating these factors, performance testing ensures that the system maintains reliable, fast, and efficient operation across diverse deployment environments.

| <b>Metric Tested</b>   | <b>Test Description</b>                  | <b>Expected Outcome</b>             | <b>Result</b> |
|------------------------|------------------------------------------|-------------------------------------|---------------|
| Inference Time         | Measure time per frame                   | Low latency (< real-time threshold) | Pass          |
| FPS<br>(Frames/Second) | Evaluate real-time processing capability | High FPS for smooth detection       | Pass          |
| CPU/GPU Usage          | Monitor resource utilization             | Efficient resource consumption      | Pass          |
| Model Optimization     | Compare before/after optimization        | Improved speed and reduced latency  | Pass          |

Table 6.5 Performance Testing

## **CHAPTER – 7**

### **CONCLUSION AND FUTURE ENHANCEMENTS**

#### **7.1 CONCLUSION**

The proposed pedestrian detection system using YOLOv8 with OpenVINO optimization has been successfully designed, implemented, and evaluated. The system effectively detects pedestrians in both static images and real-time video streams with high accuracy and consistency. By utilizing advanced deep learning techniques, the model is able to handle complex real-world scenarios such as varying lighting conditions, background clutter, and partial occlusions. The adoption of YOLOv8 as the core detection model plays a significant role in achieving efficient performance, as it combines object classification and localization into a single process. This enables faster detection compared to traditional methods while maintaining high precision. Additionally, the use of data augmentation techniques during training enhances the model's ability to generalize across diverse environments, improving its robustness. A key contribution of this project is the integration of OpenVINO optimization, which significantly improves inference speed and reduces computational overhead. This allows the system to perform real-time detection even on CPU-based systems, making it cost-effective and suitable for practical deployment without requiring specialized hardware. The experimental results demonstrate that the system achieves a strong balance between accuracy and speed, successfully detecting single and multiple pedestrians in various scenarios. While minor limitations are observed in extremely crowded scenes and very low-light conditions, the overall performance of the system remains reliable and efficient. The system not only achieves high accuracy but also maintains real-time efficiency across different hardware platforms, making it practical for deployment in real-world scenarios.. In conclusion, this project highlights the effectiveness of combining deep learning models with optimization frameworks to develop a scalable and high-performance pedestrian detection system. The proposed approach provides a solid foundation for real-world applications such as surveillance systems, traffic monitoring, and smart city solutions, and can be further enhanced with additional improvements in future work.

## **7.2 FUTURE ENHANCEMENTS**

The proposed pedestrian detection system achieves reliable performance in real-time scenarios; however, there is scope for further improvements and enhancements. Future developments can focus on increasing detection accuracy, improving performance under challenging conditions, and extending the system's capabilities for advanced applications. This section outlines potential enhancements that can make the system more robust, efficient, and suitable for large-scale real-world deployment.

### **7.2.1 Improved Detection Accuracy**

Future work can focus on improving detection accuracy in challenging scenarios such as dense crowds, extreme occlusions, and low-light environments. This can be achieved by training the model on larger and more diverse datasets and applying advanced data augmentation techniques.

### **7.2.2 Integration Of Object Tracking**

The system can be enhanced by incorporating object tracking algorithms to track pedestrians across multiple frames. This will help in applications such as surveillance, crowd monitoring, and behavior analysis by maintaining identity consistency of detected pedestrians.

### **7.2.3 Deployment On Edge Devices**

The model can be optimized further for deployment on edge devices such as embedded systems and IoT platforms. This will enable real-time pedestrian detection in resourceconstrained environments without relying on high-end hardware.

### **7.2.4 Low-Light And Night-Time Detection**

To improve performance in poor lighting conditions, future enhancements can include the use of infrared cameras or image enhancement techniques. This will allow the system to function effectively during night-time or in low-visibility environments.

### **7.2.5 Integration With Smart Systems**

The system can be integrated with smart city infrastructure, traffic management systems, and autonomous vehicles to enhance safety and automation. Features such as real-time alerts and decision support systems can also be added.

## BIBLIOGRAPHY

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016.
- [2] J. Redmon and A. Farhadi, “YOLO9000: Better, Faster, Stronger,” Proc. IEEE CVPR, 2017.
- [3] A. Bochkovskiy, C. Y. Wang, and H. Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” arXiv preprint arXiv:2004.10934, 2020.
- [4] G. Jocher et al., “YOLOv5 by Ultralytics,” 2020.
- [5] Ultralytics, “YOLOv8 Documentation,” 2023.
- [6] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite,” Proc. IEEE CVPR, 2012.
- [7] Intel Corporation, “OpenVINO Toolkit: Open Visual Inference and Neural Network Optimization,” 2021.
- [8] R. Girshick, “Fast R-CNN,” Proc. IEEE International Conference on Computer Vision (ICCV), 2015.
- [9] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection,” IEEE Trans. Pattern Analysis and Machine Intelligence, 2017.
- [10] N. Dalal and B. Triggs, “Histograms of Oriented Gradients for Human Detection,” Proc. IEEE CVPR, 2005.
- [11] P. Viola and M. Jones, “Rapid Object Detection using a Boosted Cascade of Simple Features,” Proc. IEEE CVPR, 2001.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” Proc. IEEE CVPR, 2016.
- [13] Y. LeCun, Y. Bengio, and G. Hinton, “Deep Learning,” Nature, vol. 521, pp. 436–444, 2015.
- [14] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” arXiv preprint arXiv:1412.6980, 2014.

- [15] A. Krizhevsky, I. Sutskever, and G. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” *Advances in Neural Information Processing Systems*, 2012.
- [16] OpenCV, “Open Source Computer Vision Library,” 2023.
- [17] PyTorch, “PyTorch Deep Learning Framework,” 2023.
- [18] T. Lin et al., “Microsoft COCO: Common Objects in Context,” *Proc. European Conference on Computer Vision (ECCV)*, 2014.
- [19] M. Everingham et al., “The Pascal Visual Object Classes (VOC) Challenge,” *International Journal of Computer Vision*, 2010.
- [20] W. Liu et al., “SSD: Single Shot MultiBox Detector,” *Proc. ECCV*, 2016.
- [21] S. Zhang, R. Benenson, and B. Schiele, “CityPersons: A Diverse Dataset for Pedestrian Detection,” *Proc. IEEE CVPR*, 2017.
- [22] Z. Tian et al., “FCOS: Fully Convolutional One-Stage Object Detection,” *Proc. IEEE ICCV*, 2019.
- [23] R. Joseph et al., “Darknet: Open Source Neural Networks in C,” 2013.
- [24] TensorFlow, “An End-to-End Open Source Machine Learning Platform,” 2023.
- [25] Flask, “Flask Web Framework Documentation,” 2023.